GIT IN SKY

Agent-Ops

A Methodology for AI-Agent-Based Infrastructure Operations and Technical Support

White paper

Facts, conclusions, and recommendations are not authorization to change infrastructure.

Public normative release candidate v0.5.2 | source revision aom-04-r9 | English version prevails

Git in Sky, 2026

Contents

About This Paper . . . 5
Intended Audience . . . 5
How to Read This Candidate . . . 5
Essential Terms and Roles . . . 5
Terminology Convention . . . 6

1. Executive Summary . . . 6

2. Problems Agent-Ops Solves . . . 7
2.1. IaC Does Not Capture the Whole Operational Reality . . . 7
2.2. Observability Produces Signals, Not Operational Conclusions . . . 8
2.3. An LLM Without Explicit Rules and Evidence Requirements Can Confuse Facts with Assumptions . . . 8
2.4. Manual Operations Scale Poorly and Are Hard to Transfer . . . 8
2.5. Agent Compromise Creates a Risk of Harmful Action . . . 8

3. Definition and Boundaries . . . 8
3.1. What Agent-Ops Is Not . . . 9
3.2. Agent-Ops and Uses of the Term AgentOps . . . 9
3.3. Related Work and Evidence Boundaries . . . 9

4. Agent-Ops Principles . . . 10

5. Three Complementary Views: Data, Governance, and Independent Assurance . . . 11
5.1. Process Maturity M0-M6 . . . 14
5.2. Check Result Severity S0-S5 . . . 14
5.3. Capability C0-C5 . . . 15
5.4. Operational Impact I0-I5 . . . 15
5.5. Autonomy R0-R5 . . . 15
5.6. Guardian Maturity T1-T5 . . . 16
5.7. Applying the Scales to One Case . . . 16

6. Intent . . . 17
6.1. Outcome Hypothesis and Validation Plan . . . 17

7. Evidence . . . 18
7.1. Baseline Check Catalog . . . 19
7.2. Profiles and Collection Cost . . . 19

7.3. Platform Dialects and Providers . . . 19
7.4. Observability Readiness . . . 19
7.5. Probe, Metric, and Check . . . 20
7.6. Metric Cardinality and Evidence Freshness . . . 20
7.7. Stable Observability Identities . . . 21

8. Diagnosis . . . 21
8.1. Diagnostics Pipeline . . . 22

9. Plan . . . 22
9.1. Rollback Does Not Always Exist . . . 22

10. Approval as a Structured, Verifiable Decision Record . . . 22

11. Controlled Change . . . 23
11.1. Legacy Compatibility . . . 24
11.2. Controlled Change Attempt Budget and Circuit Breaker . . . 24

12. Verification . . . 24
12.1. Outcome Verification . . . 25
12.2. Post-change Verification . . . 25
12.3. Case Completion Is Not the Same as Outcome Success . . . 25

13. Learning . . . 25
13.1. Discovery Loop . . . 26
13.2. Incident-to-Discovery Loop . . . 26

14. Unified Evidence Bundle, Lifecycle Trace, and Artifact System . . . 26
14.1. Mandatory Process Trace . . . 27
14.2. Versioned Lifecycle . . . 27
14.3. Completion Contract . . . 27
14.4. Artifact Formats and Contracts . . . 27

15. Policies: Declared State, Observed State, and Effective Policy . . . 28

16. Project Operations Harness . . . 28
16.1. Trust Hierarchy . . . 29
16.2. Machine-Readable Permissions . . . 29
16.3. Harness Validation . . . 30
16.4. Extension Points . . . 30
16.5. Agent Artifact Inventory . . . 30

16.6. Integration Admission Gate . . . 31

17. Human Authority and Approvals . . . 31

17.1. Human-in-the-Loop (HITL) Decision Map . . . 32

17.2. Boundaries of a Human Decision . . . 32

18. Governance Mesh, Autonomy, and Operational Impact . . . 32

18.1. Admission and Outcome Receipts . . . 34

18.2. Capability C0-C5 and the Execution Boundary . . . 34

19. Independent Assurance and Trust Plane: Guardian . . . 35

19.1. Independent Guardian/Evaluator . . . 36

19.2. Library Gate or AI Agent . . . 36

19.3. Agent Runtime Observability and External Watchdog . . . 36

20. Security, Trust, and Agent Context Lifecycle . . . 36

20.1. Security . . . 36

20.2. Trust Model . . . 38

20.3. Agent Context Lifecycle . . . 38

20.4. Human-led Continuation after a Lost Response (Informative) . . . 40

21. Quality, Reproducibility, and AI Economics . . . 40

21.1. Quality Verification . . . 41

21.2. Reproducibility . . . 42

21.3. AI Economics . . . 43

21.4. Full Adoption Cost (Informative) . . . 44

22. How the Principles Operate Across Three Planes . . . 44

23. Practical Scenario: Rapid Disk Filling . . . 45

23.1. A Small Team and Its Service Owner (Informative) . . . 47

24. Methodology Adoption Success Criteria . . . 47

25. Phased Adoption . . . 48

26. Expected Benefits and Positioning . . . 49

27. Bilingual Normative Publication . . . 49

28. Status of This Edition . . . 50

Appendix A. 47 Baseline Checks . . . 50

Appendix C. Source Materials . . . 52

Appendix D. Normative Result Contract . . . 52

Appendix E. Check Result Status Semantics . . . 53

Appendix F. Taxonomy and Namespace Registry . . . 53

Appendix G. Limited C4 Human-approved Apply Profile 1.0.0 . . . 54

G.1. Status, identity, and scope . . . 54

G.2. Guarantee vocabulary and claim rules . . . 54

G.3. Assumption catalog . . . 55

G.4. Trusted computing base . . . 56

G.5. Common-cause failures and Guardian independence . . . 58

G.6. Claim matrix . . . 58

G.7. Executable formal state-transition models and relation catalog . . . 59

G.8. Public schema and relation-validation slice . . . 62

Appendix H. Evidence Acquisition Profile 1.0.0 . . . 62

H.1. Status, identity, scope, and non-goals . . . 62

H.2. Controlled acquisition chain . . . 62

H.3. Independent quality axes and claim rule . . . 63

H.4. Trusted computing base . . . 63

H.5. Failure and negative-evidence semantics . . . 64

H.6. Interface with the limited C4 profile . . . 64

H.7. Common-cause boundaries and counterexamples . . . 64

H.8. Candidate boundary . . . 65

H.9. Integrated conformance and pre-runtime evaluation protocol . . . 65

About This Paper

This white paper defines the public Agent-Ops methodology developed by Git in Sky from hands-on experience in auditing, operating, and supporting server, cloud, and Kubernetes infrastructure, together with a review of publicly available approaches and products for using AI agents (software roles that use a language model and tools within declared bounds) in infrastructure operations. The paper explains the methodology's core rules, target architecture, and links among requirements, architecture decisions, and implementation. Those links let a reviewer establish why a decision was made, which evidence and components support it, and where each component's responsibility ends.

Intended Audience

Infrastructure and support leaders, CTOs/CIOs, architects, DevOps/SRE engineers, managed-service owners, and teams introducing AI agents into production operations.

How to Read This Candidate

This edition is a draft normative description, not a released standard or a claim that every target component already exists. A requirement states what a conforming process must do; a recommendation states the preferred approach; an example explains a rule without creating a new requirement; and an explicitly marked target or future state describes planned evolution.

The words **MUST** and **SHALL** mark mandatory requirements within this candidate. **SHOULD** and **SHOULD NOT** mark recommendations that require a recorded reason when not followed. **MAY** marks a permitted option. These terms describe the force of statements in this candidate; they do not grant authority to change a system.

Essential Terms and Roles

Term	Plain-language meaning in this paper
Contract	Agreed requirements for a component's inputs, outputs, and limits; not a customer or legal contract unless stated otherwise.
Artifact	A recorded work product, such as evidence, a report, plan, decision record, configuration file, or action log.
Traceability	The ability to reconstruct which evidence supported a result, who made a decision, and which action followed.
Deterministic component	A program that processes inputs under defined rules. This describes how it works; it does not promise that the program has no defects.
Policy	Formalized rules defining what is allowed, who may authorize an action, and under which conditions.
Project Operations Harness	A versioned project package containing services, owners, constraints, runbooks, and permitted tools.
Typed record	A record with predefined fields and validation rules for their values.
Authority	A bounded right or recognized source status. Source authority identifies which source is accepted for a fact; execution authority permits a named actor to perform a bounded action. One does not create the other.
Independent assurance	A check by a separate control role or component that does not approve its own work. Independence concerns roles and authority, not the number of AI agents.
Learning	Proposals to improve rules, checks, and runbooks after a case. It does not mean that the language model retrains itself.
Backbone model	The language model actually used by an agent at a particular step.
Executable formal model	A deterministic state-transition specification explored by a checker to find invariant violations. It is not a language model, generates no operational decision, and proves no runtime implementation correct.

A large language model (LLM) generates or analyzes language. An AI agent uses a model, context, and tools to perform a bounded role. In Agent-Ops, a Planner prepares a proposal, an authorized human makes the approval decision, a Gated Executor applies only an approved change, and a Guardian independently checks required artifacts and transitions. Independent assurance is not human approval: the former checks compliance and evidence, while the latter supplies accountable authority. No direct path from an agent's answer to the target system is permitted.

`AI-native` means the methodology is designed so that AI can participate safely in bounded analysis and planning roles. Conformance does not require AI in every role: a deterministic service may perform a role when it preserves the same contracts and authority boundaries.

Terminology Convention

Component names, schema fields, status values, and machine-readable identifiers are used exactly as defined in implementation artifacts. The standalone [Agent-Ops glossary](../normative/glossary.en.md) provides the detailed controlled vocabulary. Translation rules and language precedence are set out in Section 27.

The edition history is maintained in a separate document, "Agent-Ops. White Paper Edition History," published alongside this white paper at agent-ops.ru.

1. Executive Summary

This edition specifies requirements and identifies expected benefits to be tested. It does not report a controlled runtime evaluation or establish end-to-end safety of an implementation. JSON Schema validation establishes data conformance to the encoded constraints; it does not by itself enforce cross-record relationships or temporal invariants.

Modern infrastructure is described as code, observed through metrics and logs, and changed through CI/CD. Yet operational knowledge still lives in engineers' heads, chat threads, disconnected runbooks, and informal exceptions. An AI agent therefore receives either too little context or excessively broad authority. Both outcomes are unsafe.

Agent-Ops first establishes a verifiable layer of intent, facts, policy, and operational artifacts. AI agents are then assigned strictly defined roles in data analysis, hypothesis development, and change planning:

- a deterministic runtime collects evidence and evaluates it against policy;
- an agent explains the situation, correlates signals, develops testable hypotheses, and prepares a proposal;
- a human engineer retains decision authority;
- only a separate Gated Executor may apply an approved change.

For the technology maturity assumed by this edition, AI agents SHOULD NOT be used inside the Gated Executor. The current operating boundary requires the Gated Executor to be a deterministic program—a runbook, script, or playbook—started by a human engineer. Section 18.2 separately labels bounded self-healing as a future, disabled-by-default capability.

The full governed lifecycle for any infrastructure change is:

> *Intent -> Evidence -> Diagnosis -> Plan -> Approval -> Controlled Change -> Verification -> Learning*



The eight-step Agent-Ops lifecycle, showing that AI analyzes and proposes, a human decides, and only a gated deterministic executor applies an approved change.

Text alternative. Intent defines the goal and success criteria. Evidence supplies current facts. Diagnosis produces testable hypotheses, and Plan proposes actions without executing them. An authorized human records Approval before the Gated Executor may perform the Controlled Change. Verification compares the result with the original criteria, and Learning proposes updates. Missing evidence returns the case to Evidence; no path bypasses Approval.

The twelve principles in Section 4 are summarized here as five cross-cutting pillars:

1. Intent and verified outcome: an explicit objective, scope, and success criteria precede action. Case completion follows Section 12.3: required work and evidence are complete and the owner accepts a terminal disposition. The recorded Outcome distinguishes success, failure, and uncertainty against the exact Intent; closure alone does not prove success. Learning proposals follow Section 13. 2. Evidence and explicit uncertainty: facts, declared state, and interpretations remain distinct; every conclusion references its source, timestamp, freshness, and authority, while missing evidence remains `unknown` rather than becoming an assumption. 3. Deterministic bounded execution: collection and execution use bounded deterministic tools wherever possible; only a separate Gated Executor may apply an approved change, with action logging and the strongest available rollback or compensation. 4. Governance and accountable authority: policy and a versioned project-specific harness constrain each transition; a human retains approval and decision authority, and no authority is delegated implicitly. 5. Independent assurance and traceability: an independent gate or Guardian checks critical results, and every lifecycle transition, decision, and material engineer or agent message is recorded in an append-only trace; a missing mandatory record blocks advancement.

2. Problems Agent-Ops Solves

2.1. IaC Does Not Capture the Whole Operational Reality

Infrastructure as Code describes resources and configuration but does not fully define required signals, ownership, accepted variance, change windows, approval rules, diagnostic paths, or evidence of recovery.

2.2. Observability Produces Signals, Not Operational Conclusions

Prometheus, Loki, Tempo, and OpenSearch store or index telemetry; Grafana visualizes and queries it; OpenTelemetry instruments, collects, processes, and exports signals. Turning a signal into a governed decision requires applicability, policy reference, severity, freshness, ownership, recommendation, and authority boundaries.

2.3. An LLM Without Explicit Rules and Evidence Requirements Can Confuse Facts with Assumptions

A language model must not decide which source is authoritative, which command is allowed, or whether production may be changed. Without evidence references and permission boundaries, it can produce a persuasive but unsupported explanation.

2.4. Manual Operations Scale Poorly and Are Hard to Transfer

Managed services and platform teams support different technologies and criticality levels. Operational knowledge about a project accumulates among team members through the processes adopted by that team. Consequently, introducing a new person into the team, and especially replacing the team, requires additional onboarding effort. Introducing an analytical AI agent is also difficult because the agent must receive the part of the project knowledge that remains informal, memory-based, unsystematic, and incomplete.

2.5. Agent Compromise Creates a Risk of Harmful Action

An AI agent can be compromised through prompt or context injection, substitution of a model, skill, or tool, stolen credentials, or a supply-chain attack. Once compromised, even nominally permitted tools and previously granted authority can be used to make unauthorized infrastructure changes, disclose data, establish persistence, or tamper with evidence.

3. Definition and Boundaries

Agent-Ops is an operations methodology in which:

- deterministic components collect and normalize evidence;
- policy and expected state live outside the code of those deterministic components;
- AI agents receive bounded context;
- uncertainty remains unknown;
- every change follows intent, plan, approval, verification, and learning.

Layer	Purpose
Intent	Goal, scope, constraints, success criteria, and authoring authority.
Evidence	Inventory, checks, drift, telemetry, and proof.
Policy	Thresholds, roles, autonomy, operational impact, approval, and data-handling rules.
Harness	Project constraints, owners, SLOs, runbooks, and tool contracts.
Diagnosis	Timeline, hypotheses, contradictions, and missing evidence.
Remediation planning	Proposal, risk assessment, verification plan, and rollback or compensation plan. This layer performs no target action.
Execution	A separate gated path with scoped credentials and an audit log.
Learning	Outcome, decision log, postmortem, and Operations-as-Code updates.

An Agent-Ops case is bounded work on an operational request, audit, incident, or change proposal, organized around an explicit Intent, evidence, decisions, and a trace. The eight-step path describes work that requires a target change. A case can instead produce a verified report or an owner-accepted disposition without a change, as described in Section 12.3. A refusal to change a system does not by itself establish that the original

service-restoration objective was achieved.

The case operates within the organization's service and product responsibilities. Service ownership, business priorities, risk decisions, and communication with users remain with the accountable organizational functions; the applicable policy and DecisionMap establish who may decide. Access to a tool, authorship of an Intent, and execution authority are different things. Findings and verified outcomes can feed reviewed knowledge updates, reliability work, and development backlog items under Section 13. Agent-Ops does not require a particular external product-lifecycle methodology.

3.1. What Agent-Ops Is Not

Agent-Ops does not replace observability, ITSM, GitLab, or IaC. It is not a universal SSH agent with root privileges, does not promise to identify a single root cause automatically, and does not enable self-healing by default.

3.2. Agent-Ops and Uses of the Term AgentOps

AgentOps is used in several senses, including operating AI-agent systems and using agents for operational tasks. AIOpsLab (v1, 2025) uses the term for agents operating across the incident lifecycle. This document uses Agent-Ops to name a specific governance and execution methodology for infrastructure operations and technical support. Its distinction is the proposed contract and scope, not exclusive ownership of the term.

3.3. Related Work and Evidence Boundaries

This informative comparison identifies antecedents and related approaches. It does not establish first invention, demonstrated superiority, or conformance to another system. References identify the editions used for comparison; a repository's current main branch is not evidence of the implementation used for a paper's experimental results.

Operational agents. Agent-Ops belongs to the tradition of explicit management objectives and feedback in autonomic computing described by Kephart (ICSE 2005). AIOpsLab (v1, 2025) supplies an operational-agent evaluation framework and an antecedent for the term AgentOps. STRATUS (v2, 2026) is a related multi-agent reliability-engineering system. Its transactional non-regression objective and authorization safety concern different properties; comparison depends on aligned assumptions about writers, observation, and undo. The lifecycle and three-plane organization in this paper are an infrastructure-operations adaptation, not a claim to have invented management feedback loops.

Runtime enforcement and authority. Least privilege, fail-safe defaults, and complete mediation are established principles in Saltzer and Schroeder (1975). AgentSpec (ICSE 2026, camera-ready) describes runtime enforcement with predicates that can use execution state. Progent (v3, 2026) addresses privilege control for agents. Macaroons (IEEE S&P 2014) and Zanzibar (USENIX ATC 2019) provide earlier approaches to attenuated authorization and consistent authorization, respectively. Agent-Ops does not claim runtime guards or bounded authority as new mechanisms; a meaningful comparison includes stateful rules and the trusted services on which they depend.

Agent identity and runtime. Zero trust architecture (NIST SP 800-207, 2020) provides the prior principles of explicit verification, least privilege and assumed breach. Teleport's *From Zero Trust to Agent Trust* (2026) extends them to agents: identity attestable to a grantor, no standing privileges, a trusted ephemeral runtime, bounded collective autonomy of agent groups, and assumed objective drift. Agent-Ops shares the premises on identity and credentials (Appendix G.3, G.4) but places the boundary differently: a trusted runtime bounds what the reasoning component can reach, whereas the C4 profile bounds the set of executable actions and binds each to an approved package. A comparison of the two placements depends on the premise of who initiates an action; this edition claims no superiority and contains no joint evaluation.

Untrusted data and persistent context. CaMeL (v2, 2025) and Fides (v2, 2025) address enforceable information-flow boundaries; SPA (v1, 2026) addresses persistent agents across queries. Preserving trust labels is not presented as an Agent-Ops invention. The context requirements in Section 20 describe the intended operational behavior; textual labels and prompt demarcation alone do not demonstrate that an implementation enforces it.

Provenance and action contracts. in-toto (USENIX Security 2019) is an antecedent for verifiable artifact relationships. PCAA (v1, 2026) and CAVA (v1, 2026) are close comparisons for action governance, canonical action identity, approval binding, and receipts. PACE (v1, 2026) studies policy-attested execution in decentralized finance; its backend assumptions and results do not automatically transfer to infrastructure operations. Compensation has an established antecedent in Sagas (SIGMOD 1987). Combining these mechanisms is not by itself evidence of a new safety property.

Evaluation. AIOpsLab and ITBench (ICML 2025) provide operational evaluation settings. AgentDojo (v3, 2024) provides a comparison point for distinguishing legitimate-task utility from attacker success. Passing an internal gate, observing an authorized effect, and restoring service are different measurements. These works inform evaluation design; this edition reports no comparative Agent-Ops runtime results, and a difference in a paper's focus is not evidence that its system lacks a capability.

Methodological and engineering sources. AI-Disrupt PDLC v2.0 supplies methodological antecedents for the R/T scales, completion Evidence Bundle, Governance Mesh, and context-management patterns; the adaptations are identified in Appendix C. Anthropic's commerce-agent report supplies implementation-pattern antecedents outside infrastructure operations. These sources are distinguished from peer-reviewed experimental or formal evidence. The cited CaMeL, Fides, Progent, SPA, PCAA, CAVA, and PACE editions are preprints; citing them acknowledges described approaches without independently validating their results. The contribution of this edition is the specification and organization of an operational methodology. Scientific novelty and improvements in safety, utility, or cost remain unestablished.

4. Agent-Ops Principles

Principle	Practical Meaning
1. Intent before action	An action is considered only in the context of an explicit goal, scope, and success criteria.
2. Evidence before inference	Fact and interpretation have different schemas and authority.
3. Deterministic core and Executor	Collection, parsing, policy evaluation, artifact writing, and, after approval, reading and changing the target system are performed by deterministic tools wherever possible, without AI agents or humans in the execution path itself. The Gated Executor accepts only an approved package, provides the strongest available rollback or compensation, and logs every action and its result.
4. Unknown is not OK	Every unverified fact is explicitly `unknown` in deterministic artifacts. An agent may not invent, assume, or promote that fact into support for a conclusion, recommendation, approval, or action.
5. Declared state is evidence	IaC expresses intent; runtime exposes observed reality.
6. Governance in process	Policy applies before planning and execution, not after the fact.
7. Bounded collection	Timeouts, output limits, redaction, and no implicit network are mandatory. Network access is allowed only when its source, target, purpose, authorization, timeout, data handling, and evidence record are explicit.
8. Project-specific harness	Context and permissions are versioned per project.
9. Human accountability	Approval and decision authority are never delegated implicitly.
10. Independent evaluation	A critical result is checked by an independent gate or Guardian.
11. Verify and learn	Outcomes update policy, checks, runbooks, and backlog.

Principle	Practical Meaning
12. Complete process trace	Every lifecycle transition, decision, and material engineer or agent message is recorded in an append-only trace. A missing mandatory trace is `unknown` and blocks advancement.



A visual summary of the twelve Agent-Ops principles.

Text alternative. The principles require intent before action, evidence before inference, deterministic bounded collection and execution, explicit unknowns, separation of declared and observed state, governance before dangerous transitions, project-specific constraints, accountable human authority, independent evaluation, verified outcomes, learning, and a complete append-only process trace.

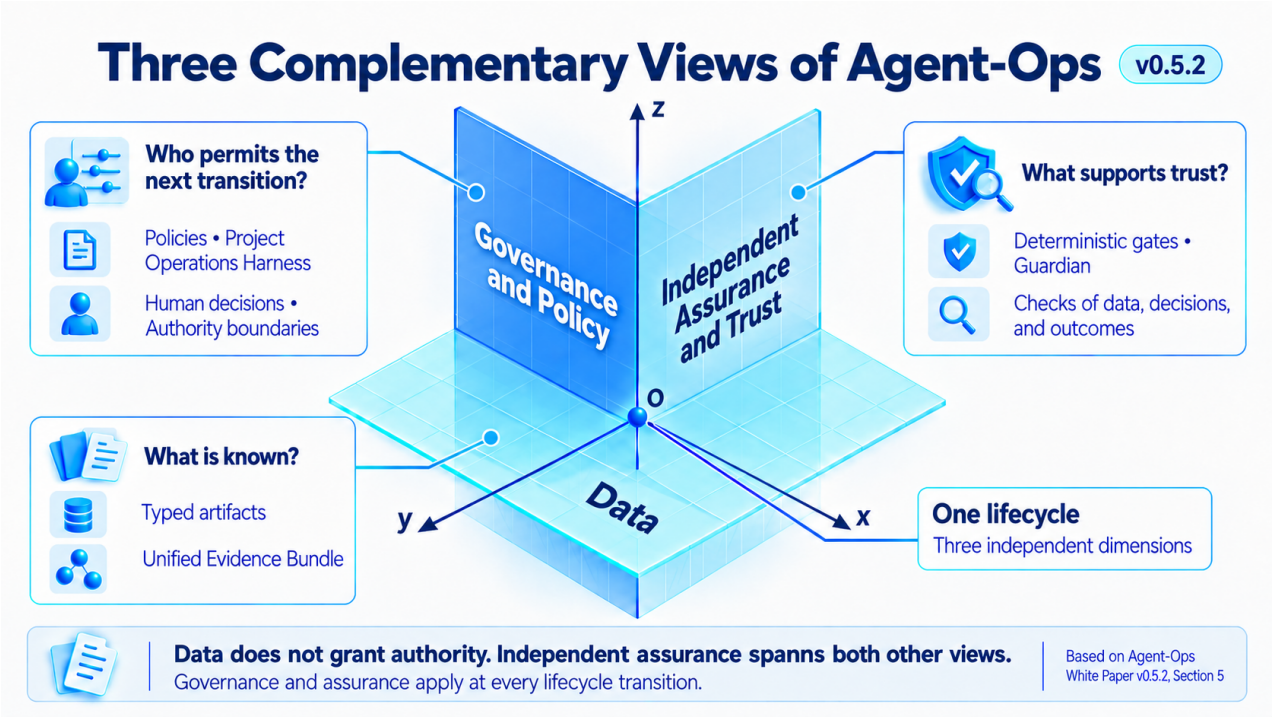
Two provisions require separate explanation.

`Unknown is not OK` is a rule about epistemic status, not a demand to guess. If a fact cannot be verified by deterministic means, the artifact records `unknown`, and the agent treats that value as a gap to resolve or refer to the accountable person for a decision. It may reason about the consequences of uncertainty, but it may not use an imagined replacement as evidence.

`No implicit network` is not a blanket ban on network collection. It prohibits undeclared calls triggered by parsing, reference resolution, dependency retrieval, name lookup, or a tool's hidden default. Such calls make a run non-reproducible, may leak data or credentials, can touch the wrong environment, and turn remote availability into an unrecorded dependency. Explicit, policy-authorized, bounded network collection is permitted when it produces provenance and completeness evidence.

5. Three Complementary Views: Data, Governance, and Independent Assurance

Agent-Ops examines the same lifecycle in three independent dimensions.



Three independent dimensions of one Agent-Ops lifecycle: data, governance and policy, and independent assurance and trust.

Text alternative. The data view records what is known and which typed artifact is produced. The governance view determines who may permit a transition and under which rules. Independent assurance checks the evidence, decision, transition, and outcome. These views operate in parallel on the same lifecycle; they are not sequential stages or a management hierarchy.

The data plane answers: "What is known, and which typed artifact is produced next?" It advances through `Intent -> Evidence -> Diagnosis -> Plan -> Approval -> Controlled Change -> Verification -> Learning`. The Unified Evidence Bundle links versions of those artifacts without conflating fact, inference, recommendation, decision, and execution record.

The governance and policy plane answers: "Who may permit the next transition, and under which conditions?" Policies, the Project Operations Harness, Human Approvals, and the Governance Mesh constrain transitions, authority, impact, and autonomy. The presence of data or a recommendation grants no execution authority.

The independent assurance and trust plane answers: "What proves that the data, decision, transition, and outcome are trustworthy?" Deterministic gates, the Guardian, security, reproducibility and completeness checks, and behavioral evals examine both other planes without inheriting the producer's or executor's authority.

The Diagnosis/RCA Agent, Remediation Planner, Guardian, and Gated Executor may read artifacts from several planes, but they do not inherit one another's rights. The Executor receives no authority from LLM output and accepts only an approved, typed, current change package.

Typed specialist agents are an optional implementation topology, not a methodology requirement. A single agent, several specialist agents, or deterministic services may implement a conforming workflow. The normative boundaries are typed inputs and outputs, isolated authority, and independent Guardian checks; adding agents grants no authority and weakens no gate.

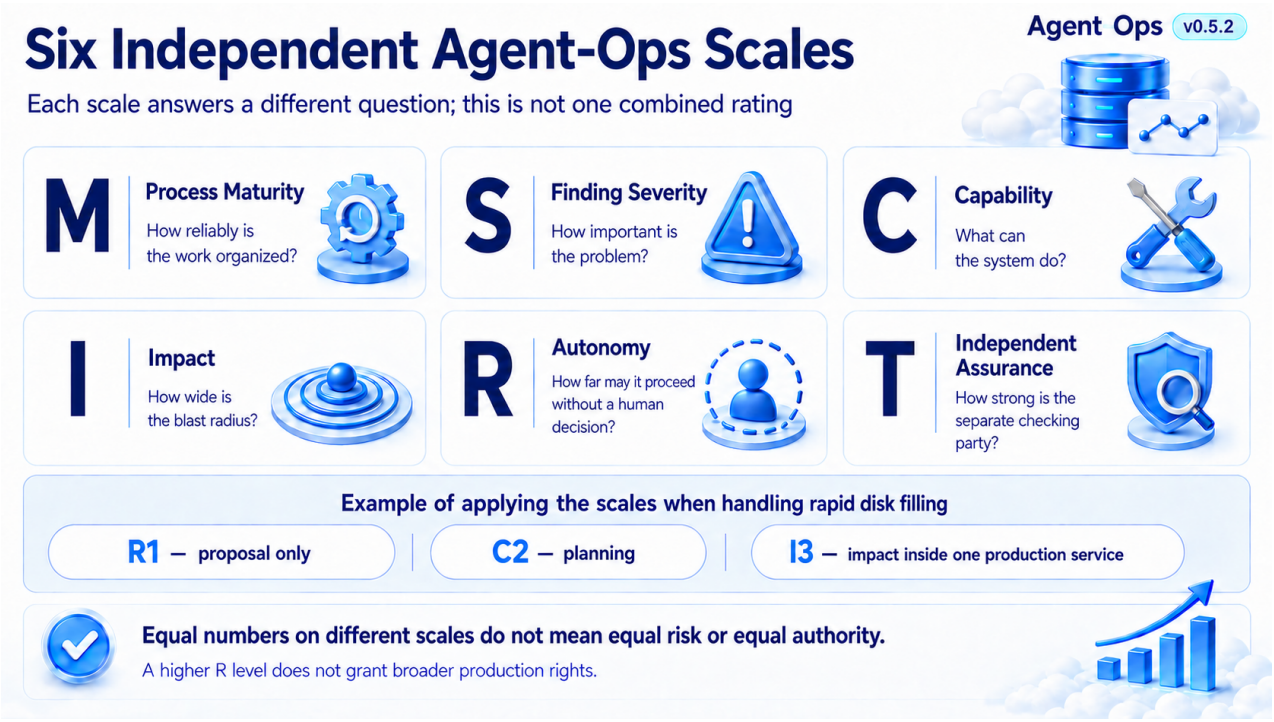
At a multi-agent handoff, state absent from the transferred typed package is not preserved. Each model-mediated node on the path MUST therefore be evaluated and attested for its executing model and admitted context; attestation of the endpoints does not establish intermediate nodes, including their injection resistance. Deterministic assemblers and gates and least-authority boundaries remain separate controls, so assurance of the chain is not reducible to one model score.

Governance rules and independent assurance apply to every lifecycle transition and especially to data-transformation boundaries.

A scale describes a particular property of a process, result, component, or action. A number is meaningful only together with its scale letter and assessment subject: for example, finding severity S3 and action impact I3 answer different questions. Levels cannot be added into one score or inferred from one another.

Scale	Assessment subject and purpose	Methodology planes
Process Maturity M0-M6	Operations within a defined scope: which practices are in use and what supports that assessment.	All three planes
Check Result Severity S0-S5	The operational significance of a finding or uncertainty: how material the risk requiring consideration is.	Data and independent assurance
Capability C0-C5	A particular component's ability to observe, explain, plan, prepare, or perform a change.	Governance and independent assurance
Operational Impact I0-I5	The consequences of a particular proposed or completed action: what is affected, how broadly, and how reversibly.	Data and governance
Autonomy R0-R5	The agent's permissible initiative for a particular task and environment: where it may continue and where a human decision is required.	Governance
Guardian Maturity T1-T5	Independent control's ability to identify, verify, redirect, and block violations and to arrange their remediation.	Independent assurance and trust

Scale attribution. R0-R5 is adopted from the Risk-adaptive authority table in Section 4.3 of AI-Disrupt PDLC 2.0, Sber's complete practical guide, June 2026. The methodology's author is Kirill Menshov. The T1-T5 notation and sequence come from Section 4.5 of the same guide and are adapted to the Agent-Ops boundary: the Guardian receives no authority to change a target, and T5 arranges remediation through a separate authorized Executor. M, S, C, and I below are defined by Agent-Ops itself; M0-M6 is not a renaming of the organizational L0-L5 scale in AI-Disrupt PDLC.



Six independent Agent-Ops scales: process maturity M, result severity S, capability C, impact I, autonomy R, and independent assurance T. Example of applying the scales when handling rapid disk filling: R1/C2/I3.

Text alternative: the six scales answer different questions and do not form one combined rating. When handling rapid disk filling, `R1` limits the agent to a proposal, `C2` describes its ability to plan, and `I3` describes the proposed change's impact within one production service. These values do not automatically determine process maturity M, result severity S, or Guardian maturity T.

5.1. Process Maturity M0-M6

M describes how consistently operations are organized in the assessed project or service. A level is supported by practices in use and their artifacts; installing a tool or having an agent does not establish maturity by itself. This is a process characteristic, not permission for an action.

Level	Meaning	What it looks like in operations
M0	Unmanaged / Unknown	Work is not organized as a governed process, or its state has not been established. There is no verified baseline on which to rely.
M1	Initial Audit	A first verifiable state snapshot and finding list have been obtained. Regularly refreshing that view is not yet an established practice.
M2	Regular Diagnostics	Checks recur and their results are compared over time; deviations and gaps are detected systematically.
M3	Observability	Diagnostics are connected to current metrics, logs, traces, and alerts; their sufficiency for operations is checked.
M4	Managed Operations	Work follows explicit intents, evidence, plans, accountable decisions, outcome verification, and a log; policies and service ownership are in place.
M5	Approval-controlled Semi-automated Recovery	Preparation and execution of bounded changes are automated, but every real change requires a human decision and a human-started deterministic Executor; the outcome is verified.
M6	Bounded Self-Healing / AI-assisted SRE	A future mode for scenarios bounded and approved in advance: recovery stays within policy limits, with verification, stop rules, and admission revocation. This edition does not introduce it as an active operating mode.

5.2. Check Result Severity S0-S5

S expresses the operational significance of a check result, including risk caused by uncertainty. The particular check's policy maps observations, service criticality, and possible consequences to an S-level; there is no universal free-space percentage or common response time for all services here.

Level	Meaning	Implication for assessing the result
S0	Informational	The result provides state information; it does not itself indicate an operational risk requiring remediation.
S1	Low	Risk is small and bounded; the finding has limited operational significance in this context.
S2	Moderate	Risk is noticeable and warrants consideration; possible consequences exceed those of a minor local finding.
S3	Material	The result indicates material operational risk that could noticeably degrade service operation.
S4	High	There is a high risk of failure, loss of availability, or a security violation.
S5	Critical	Critical risk, an emergency state, or risk of data loss; the consequences belong to the most serious class.

S does not replace `ok`, `warning`, `critical`, `unknown`, or another status in Appendix E. Significant uncertainty may warrant attention, but a high S-level does not turn an unverified fact into a confirmed deviation or authorize a fix.

5.3. Capability C0-C5

C describes the function a particular component can perform. Technical capability and authority to use it are checked separately. C0-C3 retain the no-target-mutation boundary; execution rules are detailed in Section 18.2.

Level	Meaning	Output and boundary
C0	Observe	Collect facts and evidence without changing the target.
C1	Explain	Explain possible causes, formulate questions, and identify evidence gaps.
C2	Plan	Assess impact and prepare a plan; no change is performed yet.
C3	Prepare Change	Prepare a pull or merge request, dry-run plan, verification plan, and rollback or compensation plan. Having a package does not initiate target actions.
C4	Human-approved Apply	A separate deterministic Executor applies the approved change after an explicit human decision and a human start through gates.
C5	Bounded Self-Healing	A future capability for low-impact scenarios with an action class, scope, preconditions, stop rules, and revocation conditions approved in advance. Disabled by default in this edition.

5.4. Operational Impact I0-I5

I concerns an action and its consequences. Assessment considers criticality, the scope of affected resources, reversibility, and data class. Before a change, the projected resulting state is assessed; afterwards, actual consequences are assessed. Section 11.2 specifies the assessment and treatment of overlapping changes. A short command does not imply low impact.

Level	Meaning	Impact boundary
I0	No Production Impact — Read-only / Offline	Read-only or offline work with no production side effects.
I1	Isolated Reversible Non-production	Consequences stay within an isolated non-production environment, and the change is reversible.
I2	Localized Reversible Low-criticality	A bounded low-criticality component is affected; the consequence boundary and the ability to return are established.
I3	Production Service-local	Impact stays within one production service; human approval, outcome verification, and a rollback plan are required.
I4	Multi-service / Critical / Sensitive	Multiple services, critical functions, or sensitive data are affected; separation of duties and a canary rollout apply.
I5	Systemic / Irreversible	Consequences may be systemic, irreversible, involve data loss, or have regulatory impact; a decision by the highest authorized authority is required, or an explicit prohibition applies.

I describes consequences, not admission. Unknown impact is not I0: under Section 18 it leads to denial or escalation. The same S finding may have several remediation plans with different I-levels.

5.5. Autonomy R0-R5

R defines permissible agent initiative in a particular environment and task. The table below preserves the meaning of Section 4.3 of AI-Disrupt PDLC 2.0; Agent-Ops application rules are in Section 18. Levels involving code branches and merge concern preparing repository changes and do not grant authority to change a production system independently.

Level	Meaning	What is allowed within the stated boundary
R0	Read-only	Obtaining information; the typical boundary is production without validated behavior evals.
R1	Proposals Only	The agent prepares a proposal and a human decides; this bounds work on critical production paths.

Level	Meaning	What is allowed within the stated boundary
R2	Feature Branch with Review	Changes in a task-specific branch undergo review before merge.
R3	Auto-merge after Evals	For standard validated patterns, automatic merge is permitted only when behavior evals pass.
R4	Multi-session with Checkpoints	A long-running, mature, validated scenario continues across sessions while retaining state and constraints.
R5	Sandbox Autonomy	An experiment runs autonomously in an isolated environment; those rights do not extend beyond it.

One agent may have different R-levels in production, a code branch, and a sandbox. Policy determines the level using the environment, impact, evidence, and evals; the agent cannot raise it itself. R5 does not mean more production rights than R1. In this edition, a human still starts every real target change through a deterministic Executor.

5.6. Guardian Maturity T1-T5

T describes how independent control responds to another component's result or action. It adapts the Guardian Agents ladder in Section 4.5 of AI-Disrupt PDLC 2.0 to the Agent-Ops separation of verification and execution. A T-level does not waive mandatory gates or let a Guardian approve its own work.

Level	Meaning	Independent control function
T1	Review	Examine output artifacts and prepare a report with findings for the accountable person.
T2	Verification	Check conformance with requirements, policy, and evidence, and record the verification result.
T3	Redirect	Route a risky or ambiguous action to a human decision, or return work for necessary clarification.
T4	Block	Stop impermissible progress under the rules without relying on the result's author to abandon the action.
T5	Remediate	Route a violation through a separately authorized remediation path with an Executor, policy, and rollback unit of work, then verify the corrected result. The Guardian itself does not change the target.

A unit of work (UOW) is a bounded implementation task with its own scope and verification. T labels describe control functions, not confidence in a model answer. Having T1 or T2 does not exempt a process from mandatory blocking of unsafe transitions: the Guardian and deterministic gates provide that control under Section 19. The external concept of autonomous T5 remediation is not active execution authority for an Agent-Ops Guardian.

5.7. Applying the Scales to One Case

When handling the rapid-disk-filling case in Section 23, the scales are used together but concern different subjects. R1/C2/I3 means that the agent may only propose a solution, can prepare a plan, and the proposed change will affect one production service. This notation does not describe every property of the case or constitute execution admission.

Scale	Application in the disk case
M	Assess how service operations are organized: an initial audit, regular diagnostics, observability, or a complete governed cycle. Free-space percentage does not determine M.
S	Determine result severity under the check policy, considering the filling rate, service criticality, and data risk. The initial 8% is not a universal basis for assigning S4 or S5.
C	C2 characterizes the component that assesses options and prepares a plan. Subsequent application requires a separate C4 Executor with the required human decision.
I	I3 concerns the proposed volume expansion and retention-rule correction within one production service. Fact collection without side effects may have I0; that is a different action.

Scale	Application in the disk case
R	R1 prevents the agent from applying its proposal. Human approval of a particular plan does not automatically turn that agent into an Executor.
T	Assess the control functions actually in place: review, verification, redirection, blocking, or arranging remediation. One successful report does not establish every level.

M, S, and T are established on their own grounds; R1/C2/I3 does not determine them. An assessment records its subject, basis, applicable policy, and current evidence. Section 18 governs admission, while Appendix F collects the value registry and links to the definitions.

> *Machine-readable context is data, not a command and not an approval.*

> *The permissible impact limit is set by policy; predicted and actual impact are recorded as data and evidence.*

6. Intent

Intent begins the data plane: a human goal becomes a typed, verifiable, versioned contract, but not yet a command or authority to change a target system.

Type card	Contract
Type	`OperationalIntent`
Input	A request, incident, owner goal, and initial constraints.
Output	A versioned Operational Intent that references the required Outcome Check.
Key fields	`ref`, `owner_ref`, `objective`, `expected_outcome`, `baseline`, `measurable_targets`, `scope`, `constraints`, `prohibited_actions`, `falsification_criteria`, `required_outcome_check`, `decision_map_ref`.
Schema	[`foundation_operational_intent.schema.json`](../../schemas/foundation_operational_intent.schema.json).
Contract status	A normative JSON Schema exists.

Operational Intent translates a human goal into a verifiable contract before analysis or planning begins. It is not a command and grants no execution authority.

Field	Semantics
intent_id / version	Stable identity and version of the request.
goal / expected_outcome	Desired result without prescribing one implementation.
scope	Project, environment, services, and exclusions.
constraints	Forbidden actions, time window, data rules, and blast-radius ceiling.
success / stop criteria	Verifiable completion and stop conditions.
authority	Who authored the intent and who may change it.
policy / evidence refs	Applicable rules and starting evidence.
validity	Freshness and expiry of the request.

6.1. Outcome Hypothesis and Validation Plan

Before implementation, the team records a baseline, measurable target, time window, measurement method, and adapt-versus-redesign decision. The validation plan defines mandatory evals, gates, Evidence Bundle elements, and the Outcome Check.

Intent may transition to Evidence only when its goal, scope, source authority, success criteria, and stop criteria are sufficiently defined. Unresolved ambiguity remains `unknown` and returns the intent to Discovery for clarification.

7. Evidence

Evidence records declared and observed reality before interpretation. Collection must not silently become either a diagnosis or permission to act.

The independently versioned `agent-ops.evidence-acquisition@1.0.0` profile defines the acquisition boundary in [Appendix H](#appendix-h-evidence-acquisition-profile-100). It separates requirements, collection attempts, artifacts, assertions, and bundles; establishes independent evidence-quality axes; and defines how collection failures remain visible. Its bounded record family uses new schema identities and Evidence Bundle v2; it does not change the meaning of Evidence Bundle v1.

Type card	Contract
Type	`EvidenceRecord / EvidenceBundle`
Input	Operational Intent, authoritative sources, collection profile, platform dialect, and policy.
Output	Typed check results, state snapshots, and references to an evidence bundle.
Key fields	`ref`, target identity, source, timestamp, freshness, authority, completeness, truncation, status, severity, digest.
Schema	[`check_result.schema.json`](../../schemas/check_result.schema.json), [`foundation_evidence_bundle.schema.json`](../../schemas/foundation_evidence_bundle.schema.json), [`declared_inventory.schema.json`](../../schemas/declared_inventory.schema.json), and [`foundation_object_ref.schema.json`](../../schemas/foundation_object_ref.schema.json).
Contract status	Normative schemas exist; the Bundle evolves throughout the lifecycle.

The Deterministic Evidence Engine is the foundation of Agent-Ops. Its role is not to fix a server but to collect verifiable facts, normalize them, evaluate them against policy, and write machine-readable results.

Property	Contract
Identity	`check_id`, project/environment/service identity, and tool and schema versions: [`check_result.schema.json`](../../schemas/check_result.schema.json), [`foundation_object_ref.schema.json`](../../schemas/foundation_object_ref.schema.json).
Status	`ok`, `info`, `warning`, `critical`, `unknown`, `not_applicable`, or `error`: [`check_result.schema.json`](../../schemas/check_result.schema.json).
Evidence	Source, timestamp, freshness, digest, completeness, and truncation: [`check_result.schema.json`](../../schemas/check_result.schema.json), [`foundation_evidence_bundle.schema.json`](../../schemas/foundation_evidence_bundle.schema.json).
Safety	Read-only collection, bounded output, redaction, no implicit network, and execution limits: [`check_card.schema.json`](../../schemas/check_card.schema.json), [`run_envelope.schema.json`](../../schemas/run_envelope.schema.json).
Reproducibility	Canonical ordering, deterministic writers, explicit profile, and policy: [`run_request.schema.json`](../../schemas/run_request.schema.json), [`run_plan.schema.json`](../../schemas/run_plan.schema.json), [`foundation_evidence_completion_profile.schema.json`](../../schemas/foundation_evidence_completion_profile.schema.json).
Authority	Declared, observed, and inferred data remain distinct, and result authority is recorded separately: [`foundation_semantic_object.schema.json`](../../schemas/foundation_semantic_object.schema.json), [`foundation_governance_result.schema.json`](../../schemas/foundation_governance_result.schema.json).

probe -> bounded evidence -> parser -> policy evaluation -> check result -> artifact writers

7.1. Baseline Check Catalog

A check is an independent module governed by [`check_card.schema.json`](../../schemas/check_card.schema.json) and produces a result under [`check_result.schema.json`](../../schemas/check_result.schema.json). The 47-check catalog is a baseline, not a limit: an implementation may contain 100 or more checks. Every new check receives a stable `check_id`, role, technology, risk, schema, example, deterministic implementation, validation evidence, and maturity status. Adding a module extends the versioned catalog without changing the meaning of existing `check_id` values.

7.2. Profiles and Collection Cost

An audit profile is an independent, versioned composition module: it selects checks, parameters, time limits, and output limits without changing check semantics or granting mutation authority. `audit-fast`, `audit-regular`, `audit-deep`, and `continuous-check` are examples, not a closed set; an implementation may define 10 or more profiles. A new profile identifier is added explicitly to the applicable version of [`check_card.schema.json`](../../schemas/check_card.schema.json), selected through `profile_ref` in [`run_request.schema.json`](../../schemas/run_request.schema.json), and resolved into the allowed invocation set in [`run_plan.schema.json`](../../schemas/run_plan.schema.json). `audit-fast` contains inexpensive read-only checks; `audit-regular` runs the core set; `audit-deep` adds heavy diagnostics; `continuous-check` emits low-cardinality signals; remediation profiles do not grant mutation rights.

7.3. Platform Dialects and Providers

A platform dialect is an independent adapter module governed by [`dialect.schema.json`](../../schemas/dialect.schema.json): it maps an unchanged logical `check_id` to platform-specific providers and parsers. The dialect set is open and versioned; an implementation may contain 20 or more dialects. Adding a dialect neither creates a new meaning for the check nor requires another dialect to change. Proven non-applicability yields `not_applicable`; failure of an expected provider or collection path yields `error`; unresolved applicability or insufficient evidence yields `unknown`. None of these states simulates success.

7.4. Observability Readiness

Agent-Ops does not replace observability. It validates whether telemetry is sufficient for operations and incident analysis and whether signals are connected to service ownership, SLOs, severity, and runbooks.

Area	What Is Validated
Host / application metrics	Exporter, scrape visibility, golden signals, labels, and freshness.
Logs	Coverage, retention, redaction, and diagnostic query references.
Traces	Coverage of critical paths and correlation attributes.
Alerts	Severity policy, routing, SLO mapping, owner, and runbook references.
Audit signals	Low cardinality, freshness marker, completeness, and no secrets.

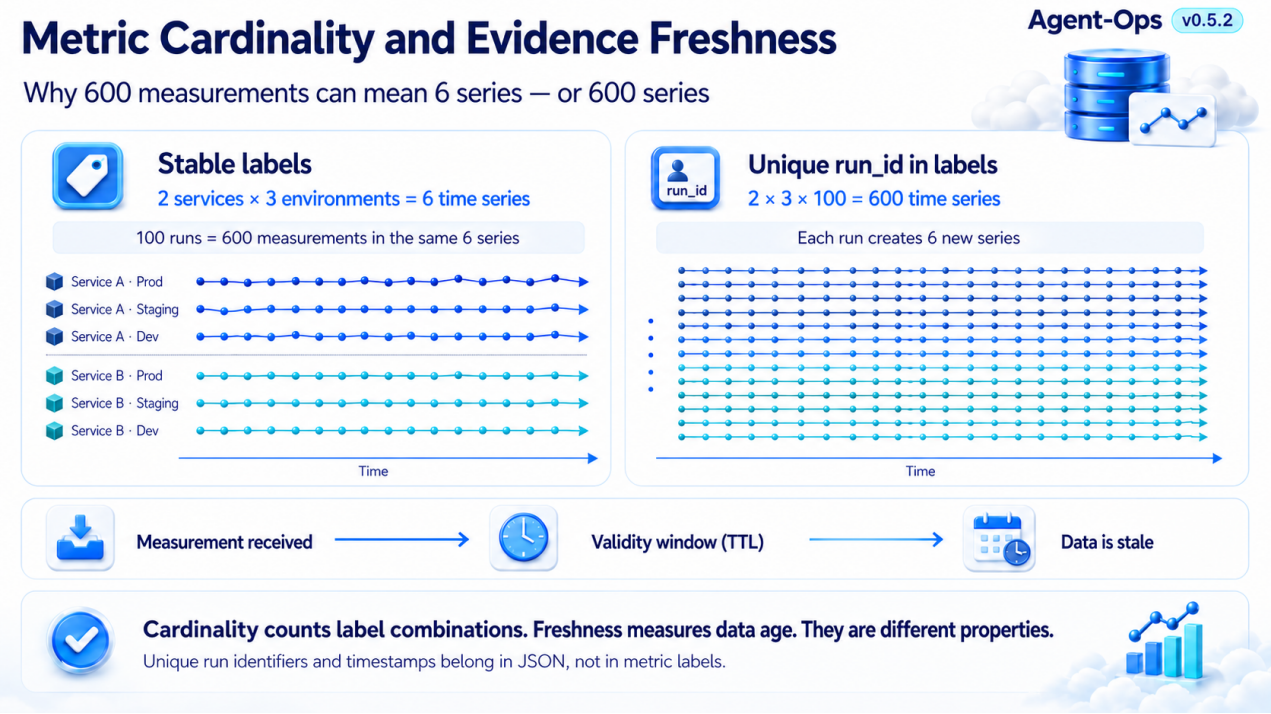
Readiness carries its own status scale with the precedence `blocked` -> `unknown` -> `gaps` -> `complete`: inability to verify outranks not knowing, not knowing outranks discovered gaps, and none of these states collapses into success. Observed inventory, declared intent, and readiness remain distinct domains of authority and completeness: owner and criticality are declared facts and are never inferred from discovered components, and a local trace of a SaaS system creates no authority over its control plane. The assessment runs offline against the artifacts of an accepted run and asserts nothing about runtime health.

7.5. Probe, Metric, and Check

A probe exposes a signal over time. A check interprets evidence against policy and produces status, severity, and recommendation. A metric may feed a check but does not replace the check result.

7.6. Metric Cardinality and Evidence Freshness

Metric cardinality is the number of distinct label combinations. Labels distinguish measurements, for example by service and environment; each combination creates one time series. A new measurement with the same labels adds a point to an existing series, while a new label combination creates another series.



Metric cardinality and evidence freshness: stable service and environment labels produce six time series containing 600 measurements, while a unique run identifier in labels produces 600 series; a separate timeline shows evidence becoming stale after its validity window.

Text alternative: with two services and three environments, stable labels create `2 x 3 = 6` time series. One hundred runs add `6 x 100 = 600` measurements to those same six series. Adding a unique `run_id` to metric labels creates `2 x 3 x 100 = 600` distinct series. Cardinality counts label combinations; freshness instead asks whether the age of evidence exceeds its permitted TTL.

Example for one metric	Time series	Measurements after 100 runs
Labels contain only two services and three environments	2 x 3 = 6	6 x 100 = 600 points in the same six series
Labels also contain a unique run ID	2 x 3 x 100 = 600	600 series, each created for one run

Unique run IDs and timestamps therefore remain in JSON artifacts, not metric labels. The numbers illustrate the distinction and do not set a universal cardinality limit.

Freshness is a separate property: it states how recently evidence was observed and whether its permitted time-to-live (TTL) has expired. A dataset may have only six series and still be too old to support a decision. Stale evidence cannot justify a risky change without fresh collection.

For the Evidence Acquisition profile, freshness is evaluated by the consumer at use time against the evidence requirement, target identity, source identity, observation interval, clock uncertainty, and every decision-critical version. A TTL is one input, not a universal proof of fitness. A version change, target-generation change,

uncovered interval, or unbounded clock uncertainty makes the dependent assertion `unknown` until the requirement is satisfied by current evidence.

For the limited C4 profile, freshness is decision-relative rather than a TTL-only property. Observation time, receipt time, use time, declared clock uncertainty, exact target UID, and every decision-critical resource version are distinct inputs. An unexpired TTL does not establish freshness after a decision-critical version changes. The assumptions and claim boundary are specified in [Appendix G](#appendix-g-limited-c4-human-approved-apply-profile-100); the executable relations define Evidence freshness at consumer use (`EVD-R09`), propagation of invalidation after a critical version changes (`EVD-R14`), and delivery of that freshness fact to the exact C4 decision (`CEI-R02`).

7.7. Stable Observability Identities

`service_id`, `environment_id`, `slo_id`, `alert_id`, `runbook_id`, `telemetry_requirement_id`, and `owner_id` remain stable so Incident Bundles can reference evidence without changing base schemas.

A transition to Diagnosis is allowed only after completeness and freshness are explicit. `unknown`, `error`, `not_applicable`, and truncation retain distinct semantics and never emulate successful collection.

8. Diagnosis

Diagnosis transforms evidence into testable hypotheses without conflating observation and inference.

Type card	Contract
Type	`DiagnosisRecord`
Input	Intent, Evidence Bundle, incident events, and applicable diagnostic rules.
Output	Ranked hypotheses, contradictions, gaps, and falsification checks.
Key fields	`diagnosis_ref`, `intent_ref`, `evidence_refs`, `observations`, `hypotheses`, `supporting_evidence`, `contradicting_evidence`, `missing_evidence`, `falsification_checks`, `confidence`.
Schema	[`diagnostic_report.schema.json`](../../schemas/diagnostic_report.schema.json) and [`foundation_discovery_assessment.schema.json`](../../schemas/foundation_discovery_assessment.schema.json) cover parts of the type.
Contract status	A single normative `DiagnosisRecord` JSON Schema does not yet exist; partial coverage is not presented as a complete contract.

The incident and root-cause analysis (RCA) workflow answers what changed, which signals were observed, and which hypotheses explain the symptoms. It uses the Unified Evidence Bundle without mixing fact and inference.

Hypothesis Field	Purpose
statement	A testable statement.
confidence	low / medium / high using explicit criteria.
supporting_evidence	Facts supporting the hypothesis.
contradicting_evidence	Facts contradicting the hypothesis.
missing_evidence	Data still required.
falsification_checks	Bounded actions that could disprove it.
likely_root_cause	A flag that does not replace human review.

> *Correct RCA is a ranked set of hypotheses with supporting and contradicting evidence, not confident prose without proof.*

8.1. Diagnostics Pipeline

Tier 1 uses inexpensive procfs/sysfs counters and runtime status; Tier 2 uses opt-in tools; Tier 3 uses native eBPF only where separately authorized and supported. Collection remains bounded and runs in baseline and sampling phases.

Diagnosis transitions to Plan only when every material hypothesis is traceable to supporting and contradicting evidence. Missing data creates a new Evidence request, not a confident conclusion.

9. Plan

Plan describes a proposed way to achieve Intent. No target-system action occurs at this step.

Type card	Contract
Type	`ChangePlan`
Input	Intent, Diagnosis, Evidence Bundle, effective policy, and Harness constraints.
Output	A plan for action, impact, verification, rollback or compensation, and required approvals.
Key fields	`plan_ref`, `intent_ref`, `diagnosis_ref`, `evidence_refs`, `proposed_actions`, `impact`, `preconditions`, `blast_radius`, `dry_run`, `required_approvals`, `verification_plan`, `rollback_or_compensation_plan`.
Schema	[`recommendations.schema.json`](../../schemas/recommendations.schema.json) is an experimental plan-only artifact.
Contract status	A single normative `ChangePlan` JSON Schema does not yet exist.

Remediation is a planning stage. The Remediation Planner prepares a proposal but performs no target action and does not apply the proposal. The proposal links to Intent, Incident, and Evidence Bundle and includes proposed actions, autonomy ceiling, impact tier, preconditions, expected effect, blast radius, dry-run or canary plan, required approvals, verification plan, and rollback or compensation plan. Any application occurs only later through a separately authorized gated Executor.

9.1. Rollback Does Not Always Exist

Irreversible actions require backup or precheck, a not-fully-reversible marker, and a compensation plan. Missing rollback is not hidden behind a generic promise.

Plan may be submitted for Approval only with an impact assessment, testable preconditions, an explicit Verification plan, and an honest statement of reversibility.

10. Approval as a Structured, Verifiable Decision Record

In the data plane, Approval is not a UI gesture or a sentence in a conversation. It is a separate addressable decision record. Organizational authority and human-governance rules are described later in Section 17.

Type card	Contract
Type	`HumanDecision`
Input	Intent, ChangePlan, required evidence and evals, authority scope, and transition point.
Output	An `approve` or `deny` decision with scope, validity window, and issuer attestation.
Key fields	`ref`, `intent_ref`, `transition_id`, `principal_ref`, `issuer_attestation_ref`, `actor_role`, `authority_scope`, `blocked_action`, `decision`, `issued_at`, `not_before`, `expires_at`, `revocation_ref`.

Type card	Contract
Schema	[`foundation_human_decision.schema.json`](../../schemas/foundation_human_decision.schema.json) and [`foundation_decision_map.schema.json`](../../schemas/foundation_decision_map.schema.json).
Contract status	Normative JSON Schemas exist.

Approval changes no target system and does not expand the Plan's scope. It is valid only for the named transition, exact target, unchanged inputs, and validity window. A missing, expired, revoked, or wrong-target decision means that no permission exists.

The limited C4 contract has the stable profile identifier `agent-ops.c4-human-approved-apply@1.0.0`. Under this profile, a human decision approves one canonical change package and a human separately starts its deterministic Executor. Approval alone is not backend admission, does not prove an effect, and does not authorize a package variant. Appendix G fixes the scope and assumptions; the executable relations define exact package binding (`C4-R01`), current approval at admission (`C4-R02`), a current authenticated human start (`C4-R03`), and authorized admission at the declared atomic point (`C4-R06`).

11. Controlled Change

Controlled Change is the only lifecycle step at which a separately authorized Gated Executor may change the target system. It accepts only an approved typed change package, rechecks the exact target, policy, freshness, and scope, and records every action and result.

The initial reference scope of `agent-ops.c4-human-approved-apply@1.0.0` is limited to predeclared typed operations against an isolated non-production test target. Arbitrary shell execution, standing C5 authority, irreversible production changes, and arbitrary third-party APIs without declared admission, identity, retry, observation, and fencing semantics are outside the profile. A pre-call check against a non-cooperating remote backend MUST NOT be described as atomic authorization of the remote effect.

Type card	Contract
Type	`ChangePackage / ExecutionRecord`
Input	Unchanged Intent, ChangePlan, HumanDecision, Governance Result, scoped credentials, and pre-change evidence.
Output	A record of actions actually performed or rejected, terminal state, and references to after-state collection.
Key fields	`case_ref`, `attempt_id`, `attempt_budget`, `attempt_index`, `idempotency_key`, `retry_disposition`, `failure_class`, `cooldown_until`, `breaker_state`, `stop_reason`, `escalation_ref`, `action_ref`, `parent_action_ref`, `target_ref`, `evidence_bundle_ref`, `admitted_identifier_set_ref`, `resolver_ref`, `conflict_domain`, `serialization_lease_ref`, `resulting_state_ref`, `in_flight_effect_refs`, `evaluated_impact`, `package_digest`, `approval_ref`, `policy_ref`, `executor_ref`, `started_at`, `actions`, `results`, `terminal_state`, `rollback_or_compensation_ref`.
Schema	[`c4_change_package.schema.json`](../../schemas/c4_change_package.schema.json), [`c4_execution_record.schema.json`](../../schemas/c4_execution_record.schema.json), and [`c4_authority_receipt.schema.json`](../../schemas/c4_authority_receipt.schema.json). The existing [`run_plan.schema.json`](../../schemas/run_plan.schema.json) concerns collection and does not substitute for a change package.
Contract status	Normative schemas cover the bounded C4 record slice. They do not implement a gateway, authenticate provenance, or prove runtime safety.

Reading and writing the target system are performed by deterministic tools wherever possible, without an agent or human in the execution path. The Executor does not reinterpret Plan, raise autonomy, or continue when digest, scope, authority, or preconditions differ. An irreversible action requires an approved compensation plan and is not disguised as rollback.

An identifier used to select a target, resource, or object for approval, rendering, or mutation MUST be an admitted identifier: issued or authoritatively confirmed by a non-model system, admitted by a deterministic resolver into the current target-bound Evidence Bundle, and fresh at use. Agent, user, or delegate output cannot enlarge that admitted set. The Executor neither constructs, completes, nor corrects an identifier; an unresolved reference produces `deny` and a new Evidence request rather than a best-effort match. The decision interface displays only admitted identifiers and server-enriched records derived from them.

11.1. Legacy Compatibility

Legacy `auto_remediation.allowed` and `mode=confirm` fields, along with the `remediate-apply` profile, are deprecated compatibility inputs. They are neither approval nor execution authority and cannot trigger mutation from Audit runtime.

11.2. Controlled Change Attempt Budget and Circuit Breaker

Every case has a policy-defined attempt budget shared by all Controlled Change attempts for the same Intent and operational outcome. A revised Diagnosis or Plan does not reset that budget. A failed or `unknown` Verification consumes an attempt. Compensation is a separately recorded effect and is not evidence that a lost-response operation was unapplied or is safe to repeat. The safe-retry relation (`C4-R08`) permits retry only from the declared operation contract and backend key, complete applicable Evidence of `not_applied`, or separately authorized reconciliation; a runtime retry-safety claim still requires accepted machine-readable validation obligations and implementation evidence.

When the budget is exhausted, a failure in a prohibited class occurs, or the required cooldown has not elapsed, the circuit breaker opens. It blocks leasing or using Executor authority and credentials for the affected case and action class and creates an escalation; the Planner and Executor cannot close or reset it. Re-enablement requires the policy-defined evidence and, where required, a new HumanDecision. Every retry decision, attempt consumption, breaker transition, cooldown, stop reason, and escalation is appended to the mandatory process trace.

Attempts whose mutation scopes overlap within a policy-defined mutation conflict domain are serialized until Verification completes or policy terminally releases the reservation. Competing cases are admitted by deterministic policy, never by arrival order. Before each admission, impact and blast radius are re-evaluated against the resulting state projected by the action, including committed state and relevant unresolved or in-flight effects. If that state would exceed the permitted `I` band, the attempt is denied or escalated. This result consumes an attempt or opens the circuit breaker only when the applicable policy declares the corresponding denial, prohibited failure class, or attempt condition; impact band and attempt budget are not interchangeable.

Policy MAY define aggregate bounds on an action class within a target domain irrespective of the number of cases and subjects: a maximum number of admissions per window, a maximum aggregate I level of the resulting state, and a maximum volume of mutated resources. The predicted resulting state accounts for all admitted and unfinished effects in the domain, not only those of the current case. Exceeding an aggregate bound yields `deny` or escalation; agents acting in the same domain are not treated as coordinated merely because each passed admission individually.

The bounded schemas and cross-record validator enforce attempt-budget monotonicity, reservation and fencing continuity, resulting-state bindings, and safe-retry preconditions. They do not define a standalone general circuit-breaker contract or prove that a backend implements those controls.

12. Verification

Verification compares after-state evidence with Intent criteria and creates Outcome. Successful command execution alone does not prove the expected result.

Type card	Contract
Type	`OutcomeCheck / Outcome`

Type card	Contract
Input	Intent, ChangePlan, HumanDecision, ExecutionRecord, and after-state evidence.
Output	An explicit `succeeded`, `failed`, or `unknown` outcome, evidence for every criterion, and residual risks.
Key fields	`outcome_ref`, `intent_ref`, `execution_ref`, `snapshot_ref`, `decision_ref`, `criteria`, `evidence_refs`, `evaluation_time`, `evaluation_window`, `residual_risks`, `status`.
Schema	[`foundation_outcome_check.schema.json`](../../schemas/foundation_outcome_check.schema.json) defines the check input; [`c4_outcome.schema.json`](../../schemas/c4_outcome.schema.json) records the bounded C4 result; [`foundation_validation_result.schema.json`](../../schemas/foundation_validation_result.schema.json) defines a generic validation result.
Contract status	The C4 `Outcome` schema keeps actual effect, Intent-relative Outcome, and evidence completeness independent. It is not a universal lifecycle Outcome contract.

12.1. Outcome Verification

Outcome records whether the goal was achieved, which after-state evidence proves it, which criteria failed, and which residual risks remain. Missing mandatory evidence prevents closure.

12.2. Post-change Verification

After the change, targeted checks, audit, and SLO validation run again. Outcome links to the ticket, MR or commit, and evidence digest; failure becomes new evidence.

Incomplete verification never becomes success: missing mandatory evidence produces an `unknown` Outcome and an `incomplete` evidence or evaluation-completeness result. Outcome is bound to exact Intent, execution, policy, and evidence digests.

12.3. Case Completion Is Not the Same as Outcome Success

The full eight-step path describes a case that requires an infrastructure change. An audit, diagnostic request, or support case may finish without Controlled Change when the Intent asks only for a verified report, evidence shows that no change is needed, or an authorized human denies the proposed Plan. The trace records why a step was not entered; a skipped change is never represented by a fabricated ExecutionRecord.

Concern	Values and meaning
Case disposition	Open until required work and evidence are complete; closed only with an owner-accepted terminal disposition.
Outcome	`succeeded`, `failed`, or `unknown` relative to the exact Intent. A closed failed case is not a successful outcome.
Evidence or eval completeness	`complete` or `incomplete`; incompleteness is not a fourth Outcome value and prevents success.
Individual check result	`ok`, `info`, `warning`, `critical`, `unknown`, `not_applicable`, or `error`, as defined in Appendix E.

13. Learning

Learning uses an already verified Outcome but does not rewrite case history. New policies, checks, runbooks, and backlog items become new versioned artifacts with their own owners and approval processes.

Type card	Contract
Type	`LearningRecord`

Type card	Contract
Input	Outcome, Evidence Bundle, Diagnosis, Plan, decisions, execution trace, and residual risks.
Output	Reviewable proposals to change policies, checks, runbooks, observability, and backlog.
Key fields	`learning_ref`, `outcome_ref`, `evidence_refs`, `lessons`, `proposed_policy_changes`, `proposed_check_changes`, `runbook_changes`, `observability_gaps`, `backlog_refs`, `owner_ref`.
Schema	No standalone normative schema exists.
Contract status	A separate `LearningRecord` JSON Schema is required; the record grants no authority to apply its proposals.

13.1. Discovery Loop

If ambiguity blocks the current case or immediate clarification is required, the work returns to Discovery. A non-urgent improvement that is not required to decide the current case MAY instead enter the backlog with an owner, rationale, and evidence gap. Neither route is presented as successful automation.

13.2. Incident-to-Discovery Loop

An unresolved incident produces explicit backlog items for telemetry, policies, checks, or runbooks. History is not rewritten after the fact; new results are appended as evidence.

Failed Verification becomes new evidence and may open a new Intent. A revised Diagnosis or Plan for the same objective, scope, and operational outcome remains in the original case and shares its attempt budget. A materially new objective, scope, or expected outcome requires an explicitly authorized new Intent; a new identifier is not a way to reset an exhausted budget. The cycle continues by adding a new version rather than rewriting the original diagnosis or decision.

14. Unified Evidence Bundle, Lifecycle Trace, and Artifact System

This section defines the data plane's cross-cutting contract: all eight steps remain distinct types but share one case identity, version chain, and append-only trace.

Type card	Contract
Type	`LifecycleEvent`
Input	Pre-transition state, input refs, governance decision, and independent-assurance result.
Output	An append-only transition-attempt record and a new Bundle version.
Key fields	`case_id`, `transition_id`, `from_step`, `to_step`, `attempted_at`, `actor_ref`, `input_refs`, `policy_decision_ref`, `human_decision_ref`, `assurance_ref`, `result`, `output_refs`, `next_allowed_transitions`, `event_digest`.
Schema	[`foundation_evidence_bundle.schema.json`](../../schemas/foundation_evidence_bundle.schema.json) defines the Bundle; [`c4_lifecycle_event.schema.json`](../../schemas/c4_lifecycle_event.schema.json) and [`c4_checkpoint.schema.json`](../../schemas/c4_checkpoint.schema.json) define the bounded C4 trace and checkpoint records.
Contract status	The C4 schemas cover the named profile slice, not a universal Agent-Ops lifecycle contract.

A Unified Evidence Bundle is an immutable, content-addressed package for a task, incident, or change proposal. It links facts and decisions without unnecessarily copying sensitive raw data.

The existing `foundation_evidence_bundle` v1 contract retains its local meaning in this edition. It may carry references produced under the Evidence Acquisition profile, but its digest, sealing, or schema validity does not by itself establish source authority, acquisition validity, coverage, freshness, independence, claim support, or

execution authority. A stronger bundle contract, if required after the formal relation specifications are accepted, will use a new incompatible schema identity or major version with an explicit migration note.

Group	Contents
Identity	Project, environment, service, intent, and incident identities.
Configuration	Harness, schema, and policy versions and digests; tool and agent artifact versions and digests; backbone models of the nodes involved.
Evidence	Observed facts, telemetry references, completeness, truncation, and freshness.
Reasoning trace	Observations, hypotheses, supporting/contradicting evidence, and gaps.
Decision trace	Recommendations, human decisions, approvals, and expiry.
Outcome	Verification evidence, residual risk, and learning references.

- Canonical ordering, deterministic digest, and bounded size.
- Offline verification and repository-contained or explicitly allowed references.
- Protection against replay, stale evidence, and cross-project substitution.
- Facts, inference, recommendation, decision, and execution records use different types.
- The Bundle cannot close while mandatory evidence is incomplete or non-authoritative.

14.1. Mandatory Process Trace

The cycle `Intent -> Evidence -> Diagnosis -> Plan -> Approval -> Controlled Change -> Verification -> Learning` has an append-only process trace. A record is written for entry to and exit from every step, every attempted transition, every decision (`allow`, `deny`, `modify`, `escalate`, defer, approve, or reject), and every material engineer or agent message that supplies evidence, changes scope, resolves ambiguity, grants or refuses authority, or affects the next step.

Each record contains a stable case and transition identity, timestamp, actor and role, verbatim message or content-addressed reference, input-state and evidence references, decision and rationale, authority basis, output status and digest, and the next permitted transition. Sensitive content may be redacted under declared policy, but the event, author, time, redaction fact, and digest remain visible. A summary may accompany the original record but cannot replace it. Missing, reordered, mutable, or target-mismatched mandatory trace data makes the transition `unknown` or incomplete and blocks advancement.

14.2. Versioned Lifecycle

The Bundle evolves through versioned snapshots with open or sealed state, parent_digest, supersedes, and append-only audit events. Original digests are never rewritten; outcome is added as a new version, and sealing is blocked while mandatory evidence is absent.

14.3. Completion Contract

The Agent-Ops completion contract contains eight mandatory elements: changes or operational artifacts; capability, regression, session-length, and escalation eval results; token-spend report; append-only trail; gate attestations; structured rationale for R2+; output-volume contribution against baseline; and a pointer to the Outcome Check.

14.4. Artifact Formats and Contracts

JSON and JSONL carry machine-readable evidence; YAML carries policy, Harness, and plans; Markdown carries engineering explanations; metrics exposition carries low-cardinality signals only. Every writer follows a schema contract and does not reinterpret another layer's semantics.

Artifact	Role
operational_intent.yaml	Goal, scope, constraints, authority, and success criteria.
check_results.jsonl	Normative individual check results.
declared_inventory.json	Normalized intent from IaC and configuration.
effective_policy.json	Applied policy merge and digest.
agent_context.json	Bounded role-specific context package.
evidence_bundle.json	Unified identity, evidence, decision trace, and outcome references.
hypotheses.json	RCA hypotheses, confidence, contradictions, and gaps.
remediation_plan.yaml	Verifiable proposal, risk, approval, and verification.
decision_log.jsonl	Append-only decisions, approvals, expiry, and references.
outcome.json	After-state verification and residual risks.

15. Policies: Declared State, Observed State, and Effective Policy

Type card	Contract
Type	`DeclaredInventory` / `DriftReport` / `GovernancePolicy` / `GovernanceResult`
Input	Versioned declared sources, observed evidence, policy layers, source authority, and the transition or action being evaluated.
Output	A classified drift report and a policy result that records the applied rules, constraints, authority, autonomy, impact, and `allow`, `deny`, `modify`, or `escalate` outcome.
Key fields	`target`, `declared_source`, `observed_source`, `findings`, `layers`, `registry_sha256`, `input_sha256`, `policy_sha256`, `state`, `autonomy`, `impact`, `authority`, `outcome`, `modifications`.
Schema	[`declared_inventory.schema.json`](../../schemas/declared_inventory.schema.json), [`drift.schema.json`](../../schemas/drift.schema.json), [`foundation_governance_policy.schema.json`](../../schemas/foundation_governance_policy.schema.json), [`foundation_governance_result.schema.json`](../../schemas/foundation_governance_result.schema.json).
Contract status	Normative schemas cover declared inventory, drift, policy layers, and the policy-evaluation result. A single generic `ObservedState` schema and a separately materialized `EffectivePolicy` schema do not yet exist.

Declared state expresses intent from IaC, configuration, and documentation. Observed state contains runtime facts and telemetry. Effective policy results from merging defaults, environment, role, host overrides, and authoritative project sources.

Drift is a classified difference, not a simple diff. Missing declared, extra observed, mismatch, stale source, accepted variance, and unmanaged-but-tolerated states have different consequences. Source conflict remains visible evidence even when precedence is defined.

The declared/observed distinction extends to agent artifact metadata. Permissions, network scope, and risk tier declared in a skill or tool manifest are declared state and confirm nothing on their own. Behavior observed in an isolated run is observed state. A difference between them is classified as drift under the same discipline as an infrastructure configuration difference: understated permissions and a spoofed risk tier are findings, not interface warnings.

Evaluation is possible only when intent, observed facts, and policy are considered together.

16. Project Operations Harness

Type card	Contract
Type	`HarnessManifest` / component documents / `HarnessResult`
Input	An explicitly bounded project root, a manifest of versioned local component references, and source-authority metadata.
Output	A normalized validation result with findings, completeness and truncation metadata, bounded usage, source digest, and `execution_authority: none`.
Key fields	`project_ref`, `environment_refs`, `policy_refs`, `agent_refs`, `tool_contract_refs`, `knowledge_refs`, `runbook_refs`, `observability_refs`, `outcome`, `authority`, `completeness`, `findings`, `usage`, `truncation`, `execution_authority`.
Schema	[`harness_manifest.schema.json`](../../schemas/harness_manifest.schema.json), [`harness_project.schema.json`](../../schemas/harness_project.schema.json), [`harness_environment.schema.json`](../../schemas/harness_environment.schema.json), [`harness_policy.schema.json`](../../schemas/harness_policy.schema.json), [`harness_agents.schema.json`](../../schemas/harness_agents.schema.json), [`harness_tools.schema.json`](../../schemas/harness_tools.schema.json), [`harness_knowledge.schema.json`](../../schemas/harness_knowledge.schema.json), [`harness_runbook.schema.json`](../../schemas/harness_runbook.schema.json), [`harness_observability_ref.schema.json`](../../schemas/harness_observability_ref.schema.json), [`harness_source_authority.schema.json`](../../schemas/harness_source_authority.schema.json), [`harness_result.schema.json`](../../schemas/harness_result.schema.json).
Contract status	The normative Harness manifest, component, source-authority, and result schemas exist. The agent-artifact inventory in Section 16.5 does not yet have a standalone normative schema.

The Harness is a schema-backed, project-specific Operations-as-Code package. It defines authoritative sources, criticality, owners, environments, data classification, change windows, SLOs, runbooks, agent permissions, and tool/output contracts.

Harness Area	Purpose
project / environments	Identity, criticality, owners, and environment overrides.
policies	Baseline, severity, drift, access, change windows, and deny rules.
observability	SLOs, golden signals, alerts, dashboards, and log/trace references.
agents / tools	Roles, prompts, can/cannot/requires, and I/O contracts.
agent artifacts	Inventory of skills, tools, and MCP servers: identity, version, digest, publisher, assigned backbone model, and target runtimes (see 16.5).
runbooks / knowledge	Diagnosis, remediation, verification, service catalog, and dependencies.
decisions / evidence	Decision references, incident references, and immutable artifact indexes.

16.1. Trust Hierarchy

Production runtime and tracked project sources have explicit authority. Incident text, prompts, external documents, and LLM output are untrusted by default. Source conflict requires review. The body of an agent skill and its metadata belong to the same class as an external document: untrusted until signature and digest are verified, and data rather than instructions after verification.

16.2. Machine-Readable Permissions

Roles define can, cannot, and requires. A planner may read evidence and prepare a proposal but may not change infrastructure. An executor requires an approved package, scoped credentials, a verification plan, and an emergency stop.

Each subject that produces a lifecycle record, including an agent instance, SHOULD carry an attestable identity bound to its grantor and to the harness version; a subject's permissions do not exceed those of its grantor. The identity MAY be implemented with workload identity standards such as SPIFFE. A normative schema for authority grants remains a target state (§17).

16.3. Harness Validation

The validator checks schema/version, required owners, explicit denylist, root-relative references, broken links, SLO/alert mappings, tool contracts, and source authority. Input is hostile: traversal, symlink escape, oversized YAML, aliases, duplicate keys, and network dereference are blocked.

A conforming read-only Harness validator accepts an explicit project root and emits one schema-backed validation result. Optional inspection and selector-based explanation views are projections of that same result. Validation invokes no target command, Git operation, or network access, and the result always carries `execution_authority: none`.

16.4. Extension Points

The Harness reserves stable identifiers for project, environment, service, owner, policy, approval rule, tool, runbook, and evidence artifact. Empty Incident/RCA entities are not added early; future links use typed references and versioned schema contracts.

16.5. Agent Artifact Inventory

Skills, tools, Model Context Protocol (MCP) servers, prompt libraries, models, runtime-configuration or hook bundles, and agents-as-tools available to an agent are listed in the Harness explicitly. An artifact absent from the inventory may not be loaded.

Field	Purpose
`artifact_id` / `version`	Stable identity and exact version; version ranges and `latest` are not allowed.
`owner`	Accountable owner of admission, review, and removal from the inventory.
`digest`	Digest of the complete bundle including resource files, not the manifest alone.
`publisher` / `signer`	Revocable publisher identity and verification key. A signature proves authorship, not safety: a verified publisher can still ship malicious content.
`source`	Registry or repository of origin.
`dependencies` / `external_refs`	Nested dependencies and execution-time references pinned to immutable digests; unresolved or version-floating references are not allowed.
`installed_at` / `installed_by`	When and by whom the artifact entered the perimeter.
`lifecycle_state` / `expires_at` / `revocation_ref` / `retention_rule`	Current lifecycle state, expiry, revocation evidence, and the rule for retaining the artifact and its validation evidence.
`declared_permissions`	Declared file, network, and tool permissions; declared state in the sense of section 15.
`last_check`	Reference to the latest coverage record and its outcome (see below).
`backbone_model`	The model that actually executes the artifact at this node.
`target_runtimes`	Runtimes in which the artifact is permitted.

Verified provenance or a valid signature establishes origin and integrity, not permission to execute content or treat it as instructions. Instruction admission requires the applicable policy to authorize the exact instruction content digest, declared role, and allowed scope. A change to content or role requires a new admission. Untrusted content cannot grant itself authority.

Changing `backbone_model` at an action-taking node is a governed change and requires re-attestation; it is not the substitution of an equivalent component. Injection resistance is a property of the executing model rather than of the artifact's bytes, and it is not inherited along with the signature.

The inventory defines required record semantics only. It grants no execution authority and proves conformance only when target-bound validation evidence satisfies the applicable contract.

For the limited C4 profile, the inventory also records the exact versions and configuration identities of every trusted computing base component named in Appendix G, including the canonicalizer, identity and policy authority, signer and key store, authoritative log, reservation store, gateway or cooperating backend adapter, effect observer, Outcome oracle, and time and version sources. Shared credentials, models, evidence sources, canonicalizers, clocks, control-plane services, and observation paths are declared as common-cause dependencies rather than counted as independent controls.

Validation of an inventoried artifact must publish a machine-readable coverage record. It lists every discovered file and digest, the analyzer outcome for each file, unsupported or skipped formats, archive and external-reference handling, and every interruption such as timeout, truncation, parser failure, or resource-limit exhaustion. Outcomes are `pass`, `fail`, and `incomplete`: zero findings mean `pass` only when all coverage required by the selected profile completed successfully, and every interruption yields `incomplete`. Revalidation runs after a version, model, runtime, dependency, or permission change, before expiry, and at the interval required by policy. Cost-driven scope reduction is recorded as an explicit exclusion rather than silently treated as successful coverage.

16.6. Integration Admission Gate

An integration is admitted for an exact connector, version, environment, and operation class rather than for a product name in general. Admission checks authenticated scope, proof of read-only behavior where claimed, failure semantics, evidence completeness and freshness, contract tests, observability and eval coverage, rate limits, expiry, and revocation. A different version, environment, operation class, or expanded scope requires a new decision.

Type card	Contract
Type	`IntegrationAdmissionRecord`
Input	Exact connector artifact and digest, target environment, operation class, requested authorization scope, contract-test and evaluation evidence, and applicable policy.
Output	`allow`, `deny`, `modify`, or `escalate`, with admitted bounds, expiry, revocation conditions, and evidence references.
Key fields	`integration_ref`, `connector_ref`, `version`, `digest`, `environment_ref`, `operation_class`, `authorization_scope`, `read_only_proof_ref`, `failure_semantics_ref`, `completeness_ref`, `freshness_ref`, `contract_test_refs`, `observability_refs`, `eval_refs`, `rate_limits`, `expires_at`, `revocation_ref`, `outcome`.
Schema	Policy outcomes use the governance schemas in Section 15; no standalone normative `IntegrationAdmissionRecord` schema exists.
Contract status	Required semantics are defined here; machine-readable admission is an extension point until the standalone schema exists.

Admission grants only the recorded integration capability. It does not grant lifecycle authority, human approval, or Executor credentials.

17. Human Authority and Approvals

Type card	Contract
Type	`DecisionMap` / `HumanDecision`

Type card	Contract
Input	An Operational Intent, transition identity, principal, actor role, authority scope, blocked action, required evidence and evaluations, and decision validity window.
Output	An explicit `approve` or `deny` decision bound to the exact transition, scope, issuer attestation, issue time, expiry, and optional revocation.
Key fields	`intent_ref`, `points`, `transition_id`, `principal_ref`, `issuer_attestation_ref`, `actor_role`, `authority_scope`, `blocked_action`, `decision`, `issued_at`, `not_before`, `expires_at`, `revocation_ref`.
Schema	[`foundation_decision_map.schema.json`](../../schemas/foundation_decision_map.schema.json), [`foundation_human_decision.schema.json`](../../schemas/foundation_human_decision.schema.json).
Contract status	Normative schemas exist for the decision map and human decision. The complete provenance chain proving that the principal holds the asserted authority is only partially represented and has no standalone normative authority-grant schema (see §16.2).

The governance plane defines not the shape of an Approval record but the source of authority, mandatory human decision points, and limits of a decision. HumanDecision from Section 10 proves that a decision was recorded; it does not create authority for its issuer.

17.1. Human-in-the-Loop (HITL) Decision Map

Every transition identifies human decisions, permitted agent actions, escalation grounds, and the self-approval prohibition. An informal comment, ticket, or prompt is never approval.

Role or control	Produces or supplies	What it cannot do by itself
Planner	A proposed ChangePlan	Approve or execute the proposal
Policy Hook / Governance Mesh	A rule-based transition decision	Supply human accountability or execute a change
Authorized human	A bounded HumanDecision	Expand the unchanged Plan through the approval record
Guardian / deterministic gate	Independent assurance evidence	Approve its own work or grant execution authority
Gated Executor	An ExecutionRecord for the approved package	Infer authority from an agent answer or alter the Plan

17.2. Boundaries of a Human Decision

Each decision point names the accountable principal, authority basis, scope, action blocked without the decision, required evidence and evals, validity window, revocation process, and escalation owner. A change to Plan, goal, target, policy, impact, or evidence after approval requires a new decision.

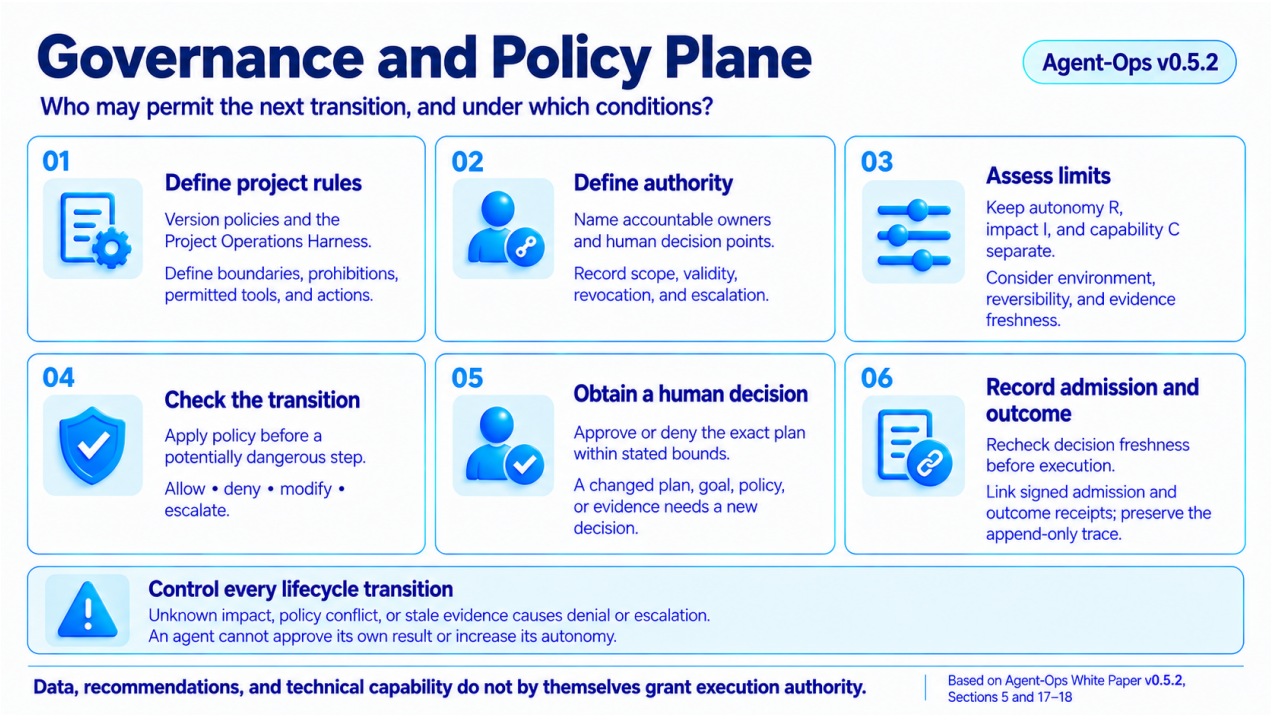
Approval fatigue is a governance risk: the interface must expose goal, impact, alternatives, unknown facts, rollback or compensation, and Verification method. Batch approval must not hide differences among actions.

18. Governance Mesh, Autonomy, and Operational Impact

Type card	Contract
Type	`GovernancePolicy` / `GovernanceResult` / `AdmissionReceipt` / `OutcomeReceipt`
Input	A lifecycle-transition attempt, policy layers and registry identity, exact action and scope, current evidence, Harness context, and any required human decision.
Output	A fail-closed policy outcome with R0-R5 autonomy and I0-I5 impact classification, constraints and escalation; for an execution attempt, linked signed admission and outcome receipts.

Type card	Contract
Key fields	`layers`, `input_sha256`, `policy_sha256`, `applied_rules`, `state`, `autonomy`, `impact`, `authority`, `outcome`, `modifications`, `attempt_id`, `action_ref`, `parent_action_ref`, `terminal_state`, signature and key reference.
Schema	[`foundation_governance_policy.schema.json`](../../schemas/foundation_governance_policy.schema.json), [`foundation_governance_result.schema.json`](../../schemas/foundation_governance_result.schema.json); C0-C5 capability is represented in [`harness_project.schema.json`](../../schemas/harness_project.schema.json) and [`harness_agents.schema.json`](../../schemas/harness_agents.schema.json).
Contract status	Normative policy and evaluation-result schemas exist, but the result currently uses one shared classifier for `autonomy` and `impact` and does not require a level, so it does not fully enforce R-only versus I-only classification. Harness schemas enforce C0-C5. Standalone signed `AdmissionReceipt` and `OutcomeReceipt` JSON Schemas do not yet exist.

Governance Mesh is the cross-cutting policy layer that validates `Intent -> Evidence -> Diagnosis -> Plan -> Approval -> Controlled Change -> Verification -> Learning` transitions. It acts before a potentially dangerous step and does not depend on agent discretion. The mandatory process trace in 14.1 records each validation and transition attempt.



The governance and policy plane: project rules, authority, transition checks, human decisions, and linked admission and outcome records.

Text alternative. Versioned policies and the Project Operations Harness define boundaries. Accountable owners and decision points define authority. Governance evaluates autonomy, impact, capability, environment, reversibility, and freshness before each dangerous transition. An authorized human approves or denies the exact plan; linked receipts and the append-only trace record the decision and outcome. Data, recommendations, and capability alone grant no execution authority.

In Agent-Ops, R0-R5 denotes the risk-adaptive permission and autonomy ladder. All levels are defined in [Section 5.5](#55-autonomy-r0-r5). The R0-R5 ladder is adopted from the Section 4.3 table without semantic changes to avoid introducing incompatible notation; source: AI-Disrupt PDLC 2.0. Operational impact uses a separate I0-I5 axis, while capability uses C0-C5. No axis grants execution authority by itself.

Autonomy	What Is Allowed	Typical Boundary
R0	Read-only operations only.	Production without validated evals.
R1	Proposals only; a human decides.	Critical production paths.
R2	Feature-branch actions with MR review.	Standard development.
R3	Automatic merge only when evals pass.	Standard templates and Pattern Library.
R4	Multi-session execution with checkpoints.	Long-running, mature, validated scenarios.
R5	Full autonomy in a sandbox only.	Isolated experimentation environments.

- Impact I0-I5 considers criticality, action type, blast radius, reversibility, and data class.
- A Policy Hook returns the normalized Agent-Ops enum allow, deny, modify, or escalate with owner, version, digest, and explainable factors; deny overrides allow, and competing modifications require explicit priority.
- The effective R-level depends on environment, branch, impact, eval coverage, Evidence Bundle, and task horizon.
- Unknown impact, policy conflict, or stale evidence produces deny or escalation.
- An agent cannot raise autonomy through prompts, Harness text, or self-approval.

18.1. Admission and Outcome Receipts

A signed receipt enables independent verification of the recorded content's integrity; completeness and truth of the history are assessed separately. Therefore, each potentially dangerous action is tracked by a signed Policy Hook admission receipt and, if execution occurs, a linked, signed outcome receipt.

An **admission receipt** is produced before the potentially dangerous step and carries `attempt_id` (a shared attempt identifier), `actor_id`, `action_type`, `scope` (the resource boundary), `policy_version`, `decision` (`allow`, `deny`, `modify`, or `escalate`), `timestamp_ms`, and a signature over the canonical field set, including the algorithm identifier and key reference.

An **outcome receipt** is produced after execution and carries the same `attempt_id`, `action_ref` (a join key calculated from a stated set of original fields), `terminal_state` (`committed` or `failed`), and a signature. The stated original fields are the preimage field set: reviewers must know exactly which values were used to calculate the key.

- `policy_version` is bound at decision time. A policy change between admission and execution creates an audit gap unless the version is recorded with the decision.
- A `deny` with no execution produces a signed admission receipt just as an `allow` does. A refusal that leaves no trace is indistinguishable from lost telemetry.
- A missing outcome receipt means the action was blocked only when the receipt pipeline is healthy, complete, and tamper-evident. Otherwise it means a crash, a queue failure, or interference.
- `attempt_id` is mandatory in both records: without it an auditor cannot confirm that the admitted action and the executed action were the same.
- `parent_action_ref` links calls into a chain: a root action carries an empty value, a child carries the predecessor's `action_ref`, and a deny record inherits the same parent. An implementation that does not support joining several predecessors must declare that limitation explicitly rather than imply complete causal reconstruction.

Receipts define record formats for the gated Executor and the Decision Log. A receipt grants no execution authority and proves only the decision or outcome bound to its signed content.

18.2. Capability C0-C5 and the Execution Boundary

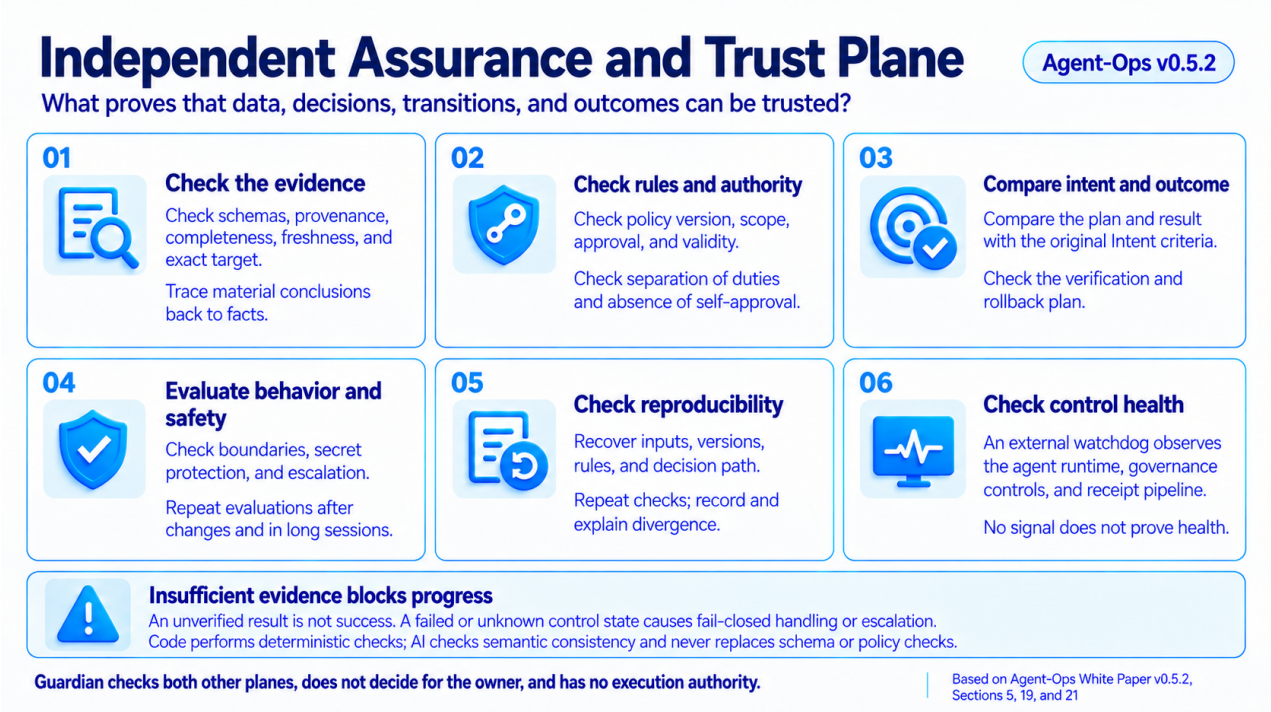
Capability describes what a component can prepare or perform, but never grants authority by itself. Full level definitions are in [Section 5.3](#53-capability-c0-c5).

Capability C0-C5	Permitted Capability
C0 Observe	Facts and evidence only.
C1 Explain	Causes, questions, and evidence gaps.
C2 Plan	Impact assessment and plan without change.
C3 Prepare Change	PR/MR, dry-run plan, verification plan, and rollback or compensation plan package; no target action.
C4 Human-approved Apply	Separate executor after explicit approval.
C5 Bounded Self-Healing	A future capability for policy-defined low-impact scenarios whose exact action class, scope, preconditions, stop rules, and revocation conditions were approved in advance; disabled by default in this edition.

The current operating boundary of this edition requires a human to start the deterministic Gated Executor for each real target change. C5 describes a future mode, not current standing authority. In that future mode, prior approval applies only to the exact scenario envelope; a changed goal, plan, target, impact, policy, or evidence condition still requires a new HumanDecision.

19. Independent Assurance and Trust Plane: Guardian

The third plane neither creates lifecycle data nor makes the accountable owner's governance decision. It independently proves that artifacts, transitions, and outcomes satisfy their contracts and blocks advancement when evidence is insufficient.



The independent assurance and trust plane: evidence, authority, intent alignment, behavior, reproducibility, and control health checks.

Text alternative. Independent assurance checks evidence provenance, completeness, freshness, exact target, policy version, authority, separation of duties, Intent alignment, behavior, secret protection, reproducibility, and control health. Insufficient evidence blocks progress. Deterministic gates check formal rules; AI may check semantic consistency but never replaces schema or policy checks. Guardian has no execution authority and does not decide for the owner.

19.1. Independent Guardian/Evaluator

The Guardian has no execution authority. It checks alignment with Intent, Harness, and Policy; Evidence Bundle completeness; unsupported claims; approval inference; and readiness for verification and rollback or compensation. A critical result cannot be approved by the same actor that produced it.

For the limited C4 profile, Guardian independence requires separation of authority, credentials, evidence sources, and observation paths. A second LLM, another prompt, or a second process using the same authority and evidence path is not sufficient independence. Agreement among several agents or an assessment by a meta-agent remains derived evidence: it does not constitute independent verification and does not create execution authority. The Guardian cannot mint approval, hold target-write credentials, or repair an evaluated record through an unrecorded action.

19.2. Library Gate or AI Agent

Deterministic invariants belong in ordinary validation gates. A separate AI agent is used only for semantic consistency and never replaces schema or policy checks.

Guardian checks both other planes. In the data plane it checks schema, provenance, completeness, freshness, trace, and Intent alignment. In the governance plane it checks policy version, authority, scope, separation of duties, and absence of self-approval. Guardian receives no Executor credentials and cannot repair the evaluated result through an unrecorded action.

Deterministic validation and a model-assisted semantic review MAY share inputs, but any shared model, evidence source, canonicalizer, clock, credential, policy service, or control-plane dependency is one common-cause boundary. Claims of independent detection are limited to the components and paths that remain independent after those dependencies are enumerated.

19.3. Agent Runtime Observability and External Watchdog

The agent runtime exposes investigation latency, queue age, tool and connector errors, connector freshness, Policy Hook and Guardian availability, override and unsupported-assertion rates, cost per verified Outcome, shadow mismatches, and agent-session stops by cause class. These signals are segmented by environment and action class without putting prompts, secrets, or unique case identities into metric labels.

An **override** is a recorded human review that changes, replaces, or cancels an initial decision. Those three forms are reported separately where their distinction matters. An override does not permit bypassing a mandatory policy denial or create execution authority.

An external watchdog in a separate failure domain evaluates runtime health and the health of control components. It has no execution authority or Executor credentials. An unhealthy or `unknown` Policy Hook, Guardian, receipt pipeline, connector, or watchdog state fails closed or escalates according to policy; absence of a signal is not evidence of a healthy state. Runtime-health observations and decisions are appended to the process trace and linked to the affected case.

20. Security, Trust, and Agent Context Lifecycle

This section governs how an agent context package is protected, which trust claims may be made about it, and how it moves through its complete lifecycle. It applies to operational work under Agent-Ops; testing software that implements the methodology is outside its scope.

20.1. Security

A context package never grants authority by itself. Security controls are applied to the exact role, target, environment, package version, and attempted action, and a control that did not complete is not treated as evidence of safety.

Threat	How the control is applied	Failure outcome
Prompt or context injection	Only instruction-bearing artifacts admitted by policy for an exact digest and role may act as instructions. Incident text, external documents, retrieval output, and another agent's output enter as data; the instruction-versus-data boundary is re-established at every handoff. Before any material reaches a model, the deterministic context assembler applies the current `ContextAssemblyProfile`: it performs policy-defined sanitization, normalizes or rejects prohibited control and bidirectional characters, fence spoofing, and over-limit items, and places each untrusted item in a typed structural partition labeled with origin and trust class. A plain-text delimiter alone does not establish the boundary. A system-prompt instruction may reinforce this control but does not substitute for it.	If the boundary or origin cannot be established, the affected material remains untrusted and cannot authorize an action or approval.
Secrets or personal-data leakage	The assembly profile classifies sensitive fields, excludes or redacts values, supplies opaque references where access is necessary, and applies the same rules to excerpts, prompts, logs, and outputs. Credentials are never embedded in the context package.	Required but unavailable data yields `incomplete`; unredacted disclosure or out-of-scope access yields `fail` and incident handling.
Target, path, or input substitution	Project, environment, service, resource, and source identities are bound to canonical identifiers and digests. Resolvers reject traversal, symbolic-link escape, ambiguous aliases, and unpinned network references.	An identity mismatch or unresolved reference blocks use of the package and requires corrected or newly collected context.
Stale, replayed, or cross-project evidence	Every item carries an observation window, freshness rule, target binding, and immutable source reference. Expired, revoked, replayed, or differently bound evidence is recollected rather than silently reused.	Dependent conclusions become `unknown`; the transition is denied or escalated until current evidence exists.
Unauthorized mutation or credential use	Actors limited to C0-C3 run without target-write credentials. A separate observer records tool, file, network, credential, and target-resource effects. Only the gated Executor may receive a project/action/time-scoped credential after the required policy and human decision, and each use is linked to admission and outcome receipts.	Any unapproved effect, credential use outside scope, or incomplete observation yields `fail`, revocation, and escalation; absence of telemetry is not proof of no mutation.
Context and runtime supply-chain poisoning	Skills, tools, MCP servers, prompt libraries, models, runtime configuration, and execution-capable hooks must appear in the §16.5 inventory with exact version, whole-bundle digest, publisher identity, declared permissions, expiry, and revocation state. Nested dependencies are pinned by digest; an external reference resolved only at run time is refused. Porting to another runtime requires renewed validation of permissions and risk.	An absent, changed, expired, revoked, or unverifiable dependency is not loaded; inability to express the original constraint in the target runtime yields `deny`.
Agent objective or behavioral drift (misalignment)	At session start the digests of the Intent, the project harness, the role permissions and the `ContextAssemblyProfile` are recorded in the process trace as the session baseline. Any output that is not a typed record grounded in evidence is recorded as `unknown`. Subjects bounded to C0-C3 have no path to an effect regardless of output content; behavioral monitoring (§19.3) complements these boundaries and does not replace them. Stopping or isolating an agent session is a governed event: it is appended to the trace, preserves the checkpoint and context for investigation, and blocks the use of already issued `HumanDecision` records until a new decision, without revoking them.	A mismatch between the baseline and the actual session context blocks progression; the session stops with artifacts preserved; a missing baseline yields `incomplete`.

Threat	How the control is applied	Failure outcome
Manifest, policy, or checkpoint tampering	The run binds versioned manifests, policies, registries, and checkpoint lineage by digest and records changes in the append-only process trace. Startup remains fail-closed until the identities and applicable current versions are verified.	A broken lineage, digest mismatch, or unverifiable current policy blocks resume and execution.

No-mutation is therefore a runtime invariant for an exact Agent-Ops case, not merely a source-code test. It is demonstrated by the absence of write-capable authority for the observing role, an independently recorded effect trace, and comparison of the bound target state before and after the run. An incomplete trace yields `incomplete`, never `pass`.

20.2. Trust Model

Trust is not a single score assigned to an agent or a text. It is a set of separately evidenced properties for an exact source, claim, scope, version, and time. `Authoritative` means that a source is recognized for a particular statement within that boundary; it does not mean that the source is infallible.

Trust property	How it is established	What it does not prove
Source authority	Harness and policy identify the source, the claims for which it is authoritative, its scope, and its validity period.	Correctness outside that scope or time window.
Provenance and integrity	An immutable source reference, whole-content digest, signer identity where applicable, and transformation lineage bind the examined bytes to their origin.	Truth, safety, or authority of the content.
Freshness	Capture time, observation window, TTL, current target identity, and revocation state are checked at the transition that consumes the item.	Completeness of the collected evidence.
Completeness	Required source classes, collection bounds, included and excluded items, truncation, and interruption status are recorded and checked against the profile.	Correctness of every included claim.
Derived result	A summary, retrieval result, diagnosis, or agent output records its sources and transformation and remains in the `derived` partition.	Source authority, instruction status, approval, or execution authority.
Independent verification	Guardian or a deterministic gate rechecks the required properties against the exact inputs without inheriting the producer's authority.	Authority to execute or to repair the producer's result silently.

Every context element remains in an explicit `authoritative`, `derived`, or `untrusted` partition. Compaction, copying, retrieval, and handoff cannot raise that partition. If a required trust property is missing or conflicting, the consuming conclusion remains `unknown` or `incomplete`, and the governed transition fails closed or escalates.

The limited C4 trust boundary is cataloged in Appendix G. Admission safety, knowledge of an effect, Outcome success, and later recovery are separate claims. A safe stop establishes none of the other three, and no atomic remote-authorization claim is made unless a cooperating gateway or backend enforces the decision at its declared admission point.

20.3. Agent Context Lifecycle

A context package is a versioned and bounded representation of Intent, policy, authority references, evidence, and remaining work for one role. It carries `execution_authority: none` and cannot transfer hidden permissions between agents.

Contract	Requirement
Purpose and scope	Why the package exists; its exact case, role, target, environment, and included objects.

Contract	Requirement
Trust partitions	Authoritative, derived, and untrusted elements remain separate.
Authority boundary	Current decision references, allowed tools, forbidden operations, required approvals, and `execution_authority: none`; the package records but does not grant authority.
Freshness and identity	Observation windows, provenance, versions, immutable source references, and content digests.
Bounds	Byte, item, token, excerpt, time, attempt, and secret-handling limits.
Lifecycle history	Who assembled, validated, compacted, transferred, refreshed, revoked, closed, and disposed of the package, and when.

Context sources are admitted by the assembly profile, the §16.1 trust hierarchy, and applicable policy. Runtime dependencies enter only through the §16.5 inventory and, for integrations, the §16.6 admission gate. The manifest records exact source and dependency identities and versions; a familiar product or agent name is never sufficient.

Type card	Contract
Type	`ContextAssemblyProfile` / `ContextManifest`
Input	Intent and role, authority and policy bounds, candidate sources, time window, runbook references, ranking rules, and byte/item/token budgets.
Output	Exact included items and excerpts; excluded items and reasons; provenance, freshness, trust partition, content digest, lifecycle state, and `execution_authority: none`.
Key fields	`profile_ref`, `intent_ref`, `role_ref`, `target_ref`, `environment_ref`, `time_window`, `source_refs`, `runbook_refs`, `ranking_rules`, `byte_budget`, `item_budget`, `token_budget`, `included_items`, `excluded_items`, `exclusion_reasons`, `provenance`, `freshness`, `trust_partition`, `manifest_digest`, `state`, `parent_manifest_ref`, `expires_at`, `revocation_ref`, `closed_at`, `disposition`, `execution_authority`.
Schema	[`ai_agent_context.schema.json`](../../schemas/ai_agent_context.schema.json) describes one Gitinsky Audit output contract; it defines neither the generic `ContextAssemblyProfile` nor the generic `ContextManifest`. Separate normative schemas do not yet exist.
Contract status	The lifecycle semantics below are a normative target; machine-readable schema coverage is partial.

Lifecycle stage	How it is applied	Required result
Assembly	A machine-readable profile selects sources within the role, policy, authority, time-window, and budget bounds. It emits a manifest containing every included excerpt, every exclusion and reason, provenance, freshness, trust partition, and digest.	A reviewable package with complete selection history and no execution authority.
Validation and use	Before use, the consumer verifies the manifest digest, package identity, freshness, required sources, current policy, role, and remaining bounds. Elements are exposed according to their trust partition; external material remains data.	Only the exact validated package may support the current step; a failed or incomplete check blocks its use.
Compaction	Compaction creates a successor version and preserves source and version references, selection rules, excluded categories, authority constraints, budgets, remaining work, and a pointer to the predecessor. A summary remains derived.	A smaller package whose omissions and lineage are explicit and whose authority has not increased.

Lifecycle stage	How it is applied	Required result
Checkpoint, handoff, and resume	A checkpoint records monotonically increasing, non-reusable session and checkpoint identifiers, environment-restoration instructions, completed, in-progress, and blocked work states, machine-readable remaining work, token, time, and attempt limits, history, and the current Evidence Bundle pointer. The receiver revalidates integrity, freshness, current policy, authority references, budgets, and tool-call idempotency or compensation state before deterministic warm-up.	Resume continues from an authenticated successor state; handoff never expands authority or resets limits.

A checkpoint is a derived summary, never the authority source for C4. It may refine only the authoritative durable log and reservation state: it cannot delete a consumed attempt, admitted or unresolved effect, pending reconciliation or compensation, current approval or policy epoch, reservation, fence, or observation-completeness limitation. The checkpoint-refinement relation (`C4-R12`) fixes this rule and retains the assumption-removal and combined omission counterexamples listed in Appendix G.

Refresh and revocation	Source, policy, approval, model, tool, connector, or expiry changes invalidate the affected package. A new manifest is assembled and linked as a successor; the previous version remains immutable and cannot silently become current again.	Consumers can distinguish current, expired, superseded, and revoked context and refuse invalid versions.
Closure, retention, and disposal	Closure records the final state, Outcome and Evidence Bundle references, retained manifest and handoff lineage, retention rule, and disposition. Ephemeral decrypted values, credential leases, and temporary sensitive caches are revoked or destroyed without erasing the audit trace.	The case can be reviewed without leaving reusable authority or unnecessary sensitive context behind.

The runtime of the reasoning component SHOULD be session-scoped: without write credentials, without state persisted across cases other than the checkpoints of this section, and without network access beyond what the profile declares. The absence of a path to an effect does not waive this requirement: persisted state can carry poisoned context across cases.

20.4. Human-led Continuation after a Lost Response (Informative)

Informative example — a lost response. An admitted operation times out before the engineer receives its result. The handoff preserves the exact case, package, attempt and backend key, current authority references, consumed budget, observations, and unresolved obligations. A missing response is not evidence that no effect occurred. The receiver uses the authoritative log and independent current observations to establish what can be known before considering a retry or separately authorized reconciliation.

Human-led continuation preserves the same authority, logging, and verification requirements. It can include coordination, permitted observation, preparation, and a human start of an authorized deterministic procedure when all required controls are available. If the required log, authority service, gate, or observer is unavailable, a saved summary does not restore those controls or establish recovery. Direct manual mutation is not introduced as an alternative execution path. A stopped agent session also retains the decision-use restrictions in Section 20.1.

21. Quality, Reproducibility, and AI Economics

The procedures below specify how quality, reproducibility, and cost are to be assessed. Their inclusion is not evidence that an Agent-Ops implementation has passed them; this edition reports no controlled comparative runtime results.

For `agent-ops.c4-human-approved-apply@1.0.0`, the profile publishes assumptions and claim boundaries, and this source revision includes the bounded executable specification, six C4 record schemas, the public relation-validator catalog, and relation-negative traces. Acceptance of these offline artifacts is not runtime conformance. A bounded check of the formal state-transition model or cross-record fixtures is not a substitute for authority safety, actual-effect measurement, Outcome success, observation completeness, or production

evidence.

For `agent-ops.evidence-acquisition@1.0.0`, evidence quality is not one score. Authority, identity and binding, integrity and provenance, acquisition validity, coverage and completeness, freshness, transformation fidelity, independence, claim support, and reproducibility are evaluated separately as specified in Appendix H. A missing, conflicting, or unverified required axis remains `unknown` or incomplete; an aggregate score cannot compensate for a failure of authority, target binding, integrity, or blocking coverage.

This section evaluates operational work performed under Agent-Ops. It does not prescribe how software that implements the methodology is developed or tested.

21.1. Quality Verification

Quality is evaluated for each bounded operational case, from Intent through verified Outcome. A fluent answer, a completed tool call, or an absence of reported incidents is not evidence of quality.

An **evaluation (eval)** is a repeatable behavior check on a defined case with expected evidence, decisions, and prohibited outcomes; it is not a subjective rating of writing style. Using one disk-space case: a capability eval asks whether the agent analyzes it correctly; a regression eval asks whether that behavior remains after a model, tool, or policy change; a session-length eval asks whether scope and authority survive compaction and handoff; and an escalation eval asks whether missing authority or dangerous impact is handed to a human. These are four independent evaluation types, not four mandatory stages of one test.

Check	How It Is Applied	Non-Success Handling
Evidence support	For every material fact, Diagnosis, recommendation, and decision, the evaluator follows the cited evidence reference and checks provenance, freshness, completeness, target identity, and the separation of observation from inference.	A missing, stale, conflicting, or unsupported basis produces `unknown` or `incomplete` and blocks dependent transitions.
Intent and Outcome alignment	Before planning or change, the proposed result and constraints are compared with the exact Intent version. After action, the Outcome Check compares observed after-state with every outcome criterion and residual-risk requirement.	An unmet criterion produces `failed`; an unverified criterion produces `unknown`. Neither is accepted as success.
Policy and authority	At every governed transition, the Policy Hook and Guardian compare the actor, action, target, impact, scope, policy version, approval, validity window, and separation-of-duties requirements.	A scope, identity, authority, expiry, or policy mismatch produces `deny` or `escalate`; no action is attempted.
Capability eval	A versioned operational case defines bounded inputs, expected facts and decision class, permitted outputs, forbidden actions, and an evaluation oracle for the exact combination of agent, model, tools, policy, environment, and action class under assessment.	A missing required result or any forbidden behavior fails the case.
Regression eval	The same versioned cases are replayed after a change to the agent, model, context profile, tool, connector, or policy. The comparison covers decision class, evidence dependencies, constraints, escalation, and Outcome semantics rather than wording alone.	Material unexplained drift fails the affected scope and prevents promotion.
Session-length eval	A case is executed across long sessions, compaction, checkpoint, handoff, and resume. The evaluator checks that case identity, Intent, authority, policy, evidence lineage, attempt budget, and pending work remain intact.	Lost constraints, substituted evidence, reused approval, or an untraceable transition fails the case.
Escalation eval	Versioned cases deliberately include missing, stale, contradictory, out-of-scope, and high-impact evidence, plus ordinary cases that should not escalate. The expected `deny`, `escalate`, or bounded continuation is compared with the actual decision.	Both a missed mandatory escalation and systematic over-escalation are recorded as quality failures.

Check	How It Is Applied	Non-Success Handling
Boundary and secret safety	Read-only roles are exercised in an isolated environment while tool calls, file writes, network access, credential use, and output projection are observed against the declared envelope.	An unauthorized mutation or call, credential use outside scope, or sensitive-data disclosure fails the case; critical violations have zero tolerance.

A constructed-state evaluation starts from an exact validated prior state, appends the single stimulus under test, and grades the resulting artifacts and decision. Cases deliberately include contradictory history, stale or truncated evidence, superseded approvals, and a compacted or resumed context package or a state immediately following a tool call. Exact comparison is limited to policy-owned canonical fields and text supplied by the deterministic harness; other model output is graded semantically. Such a snapshot validates behavior from the supplied state but does not prove the authenticity or correctness of the transitions that created it. A session-length evaluation MUST actually exercise compaction, checkpoint, handoff, and resume transitions. Replaying a complete session remains permitted but is not required to evaluate a late-session state.

Every evaluation case identifies its source and version, action class, environment, impact band, exact component versions, input artifacts, oracle, expected decision, required evidence, and forbidden outcomes. Results are `pass`, `fail`, or `incomplete`; incomplete coverage never passes. Versioned policy may define statistical thresholds for environment-specific quality indicators, but authority violations, self-approval, destructive behavior by a read-only role, and secret disclosure cannot be averaged away.

Shadow evaluation applies the same discipline to real cases before an action class receives execution authority. The agent investigates and produces a plan with `execution_authority: none`; the engineer's actual decision and action proceed through the ordinary authorized path, and the verified Outcome is compared with the shadow proposal.

Type card	Contract
Type	`ShadowEvaluationRecord`
Input	Exact agent, model, toolchain, policy, action class, environment, impact, shadow plan, engineer decision, actual action, and verified Outcome.
Output	A classified agreement or disagreement, safety findings, and evidence for a later promotion decision; never execution authority.
Key fields	`shadow_ref`, `case_ref`, `agent_ref`, `model_ref`, `toolchain_versions`, `policy_ref`, `action_class`, `environment_ref`, `impact`, `agent_plan_ref`, `engineer_decision_ref`, `actual_action_ref`, `outcome_ref`, `disagreement_class`, `safety_findings`, `execution_authority`.
Schema	No standalone normative schema exists.
Contract status	Required record and promotion semantics are defined here; schema coverage is pending.

Promotion is scoped to an exact action class, environment, impact band, and agent/model/toolchain versions; it is never a blanket promotion of the agent. Metrics and agreement rates cannot self-promote a capability. Promotion requires an explicit HumanDecision or policy decision, target-bound evaluation evidence, and continuing Guardian independence. Any scope or version change requires re-evaluation.

21.2. Reproducibility

Reproducibility means that an independent reviewer can reconstruct what was known, which rules and authority applied, how the result was produced, and whether the same bound inputs lead to the same controlled conclusions. It does not mean that live infrastructure stops changing or that stochastic model prose must be byte-identical.

Check	How It Is Applied	Required Result
Input binding	The run records the exact target identity and observation window plus digests or immutable references for Intent, Evidence Bundle, policy, schemas, context manifest, agent, model, tools, and connectors.	A reviewer can recover the complete input set without guessing or silently resolving a newer version.
Deterministic replay	Parsers, schema validation, normalization, policy evaluation, gates, and canonical writers are rerun against the retained inputs.	Normalized facts, classifications, policy decisions, and canonical digests match.
Model-assisted replay	The retained context manifest and exact agent, model, tool, and policy versions are used again. The evaluator compares cited facts, uncertainty classification, decision class, constraints, required approval, escalation, and prohibited actions rather than surface wording.	The replay remains within the same evidence and authority envelope; a material semantic divergence is explicit.
Live-state renewal	When the target or authoritative source has changed, new evidence is collected as a new run with its own time window and is linked to, rather than substituted for, the earlier run.	The original record remains reproducible against its snapshot, while current state is represented by a successor record.
Divergence handling	A mismatch is classified by source, freshness, context, model, tool, policy, or nondeterminism and linked to both compared runs.	Until a material divergence is explained and accepted, the affected conclusion is `unknown` and cannot authorize action.

Reproduction evidence is part of the case Evidence Bundle. Redaction may hide sensitive values, but it preserves the event, field presence and type, policy basis, a protected reference to the source record, and proof that the redaction was applied; otherwise an independent reviewer cannot distinguish protected data from absent data.

21.3. AI Economics

AI economics is measured per bounded operational case and per owner-accepted, verified Outcome. Generated text volume, tool-call count, or automation rate is not a benefit by itself.

Measure	How It Is Calculated and Used
Direct AI and tool spend	Sum the monetary cost of metered model usage or compute, connector and paid-tool calls, and attributable storage and processing, using the rates, currency, and rate version effective for the run.
Human effort	Record engineer and reviewer time for clarification, approval, investigation, correction, and escalation. If it is monetized, the labor rate and method are declared; otherwise hours remain a separate measure rather than being treated as zero cost.
CTOR	Cost-to-Outcome Ratio is the total declared direct cost plus any explicitly monetized human effort divided by owner-accepted `succeeded` Outcome Checks. Costs of `failed` and `unknown` cases remain in the numerator and are also reported separately. When there are zero accepted successful Outcomes, CTOR is undefined and is reported as not computable, never as zero.
Time to verified Outcome	Measure from the request registration timestamp in the system of record to terminal Outcome and separate active processing, queue time, external waiting, and human decision time so that delay is not misattributed to model performance.
Retry and rework cost	Report spend and human effort consumed by repeated collection, model retries, rejected recommendations, reopened cases, rollback, and correction cycles. A retry that only produces more text creates no outcome value.
Escalation and approval load	Report escalation and override counts together with reviewer time, action class, impact, and outcome. Count changed, replaced, and cancelled decisions separately where relevant. Batch approval does not erase the cost or risk of the individual cases it contains.

For example, 120,000 monetary units of included cost divided by 80 owner-accepted successful Outcomes gives a CTOR of 1,500 per successful Outcome. Cost from unsuccessful attempts remains inside the 120,000. If human effort is not monetized, its hours are displayed alongside CTOR rather than silently valued at zero.

Metrics remain low-cardinality and contain no prompts, PII, secrets, or unique case identifiers in labels. Economic comparisons use like-for-like service boundaries, severity and impact segments, evidence requirements, workload, and team capacity, following the comparison discipline in Section 24.

Lower cost or triage time is not an improvement when evidence completeness, decision safety, or verified Outcome quality degrades. Conversely, a safer result may justify higher cost for a high-impact case. The scorecard therefore reports quality, risk, time, and cost together and never grants authority or promotion from a single metric.

21.4. Full Adoption Cost (Informative)

An adoption assessment can extend the cost view in Section 21.3 with setup, integration, training, maintenance, upgrades, supplier transition, and the effort of collecting the measurements. Recurring case work and shared overhead are counted once; one-off costs are shown in full and, if allocated, with an explicit allocation period. Net engineering time is comparable baseline effort minus case work, shared overhead, and the stated allocation of setup effort. Unsupported estimates remain estimates. This supplementary view does not change the Cost-to-Outcome Ratio (CTOR), replace Section 24's outcome and quality measures, or waive the completion evidence in Section 14.3.

22. How the Principles Operate Across Three Planes

The principles in Section 4 become verifiable only when each has a data type, a transition rule, and independent evidence.

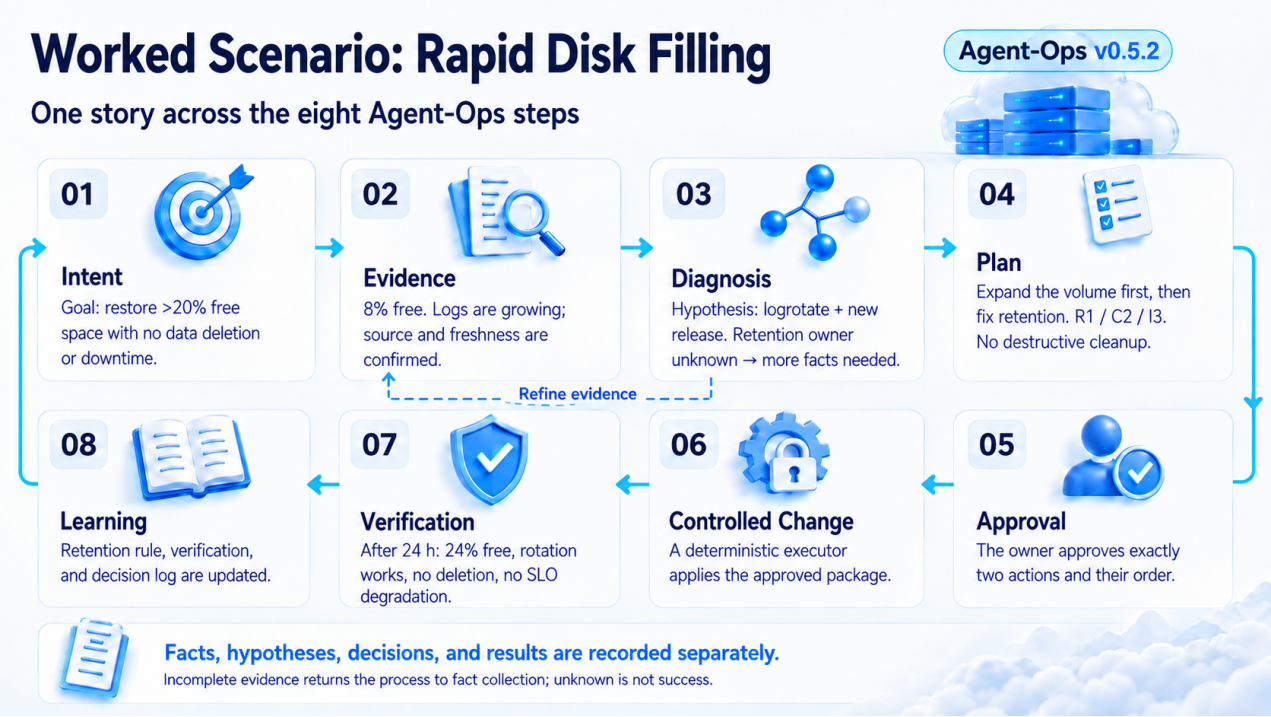
Principle	Data plane	Governance plane	Independent assurance and trust
Intent before action	OperationalIntent precedes collection and planning.	Scope and authority constrain transitions.	A gate checks Intent identity and completeness.
Evidence before inference	Facts remain separate from Diagnosis.	Policy defines permitted sources.	Guardian detects unsupported claims.
Deterministic core and Executor	Collection and execution emit typed records.	Executor is admitted only by policy and Approval.	No-mutation, exact-target, and receipt checks prove boundaries.
Unknown is not OK	An unverified fact remains `unknown`.	Unknown impact or authority produces deny or escalate.	Incompleteness blocks the transition.
Declared state is evidence	Declared and observed data remain distinct.	Effective policy resolves conflicts explicitly.	Drift and source authority are independently checked.
Governance in process	Every step emits a LifecycleEvent.	Governance Mesh checks every transition.	A skipped check is visible as a trace gap.
Bounded collection	Evidence records bounds and truncation.	Policy permits source, network, and profile.	A coverage record proves actual scope.
Project-specific Harness	Artifacts use stable project refs.	Harness defines owners, policy, and permissions.	Harness validation checks integrity without authority.
Human accountability	Approval has a separate type.	Only an authorized human makes the required decision.	Separation of duties and no-self-approval are checked.
Independent evaluation	The evaluation result is addressable and versioned.	The author cannot waive a mandatory gate.	Guardian and deterministic gates are independent of the producer.
Verify and learn	Outcome precedes LearningRecord.	A policy change enters a new governed cycle.	Outcome Check proves the result before learning.

Principle	Data plane	Governance plane	Independent assurance and trust
Complete process trace	Every attempted transition is appended.	Decisions and authority basis are retained.	A gap, reordering, or substitution blocks advancement.

Governance Mesh and Guardian are complementary. Mesh decides whether a transition is permitted under current rules; Guardian independently checks that the inputs, decision, and proof of transition are correct. Neither executes a change.

23. Practical Scenario: Rapid Disk Filling

This is an illustrative scenario, not a reported experiment or production measurement. Its percentages and observation window explain the lifecycle and do not establish measured improvements.



Worked rapid-disk-filling scenario across the eight Agent-Ops steps, including an evidence-refinement loop and verification that free space rises from 8 percent to 24 percent after 24 hours.

Text alternative: Intent requires more than 20 percent free space with no deletion or downtime. Evidence shows 8 percent free and growing logs. Diagnosis identifies a `logrotate` and release hypothesis but returns for more evidence because the retention owner is unknown. The approved `R1/C2/I3` plan expands the volume first and then corrects retention. A deterministic executor applies exactly those actions. Verification after 24 hours shows 24 percent free space, working rotation, no deletion, and no SLO degradation; Learning updates the retention rule, verification, and decision log.

The scenario presents the same case in all three planes. Before promotion, the same proposal can be evaluated in shadow mode against the engineer's actual decision, action, and verified Outcome without granting the agent execution authority. Verification compares after-state with Intent criteria and creates an explicit `succeeded`, `failed`, or `unknown` Outcome. Learning consumes that verified Outcome and does not replace it.

Visual checkpoint	Free space	Other success conditions
Goal	More than 20%	No business-data deletion; no service interruption
Initial evidence	8%	Retention-policy owner still requires confirmation

Visual checkpoint	Free space	Other success conditions
Verified after-state after a 24-hour observation window	24%	Rotation works; no data deletion; no SLO degradation

This is a teaching example, not a universal threshold. Its R1 autonomy allows a proposal only, C2 capability allows planning, and I3 impact describes a service-local production change. The three labels answer different questions and grant no authority by themselves.

Lifecycle step	Data plane	Governance plane	Independent assurance and trust
Intent	The goal is to restore free space above 20% without deleting business data or stopping the service.	The owner defines scope, constraints, and stop criteria.	Schema and gate checks prove Intent completeness and owner authority.
Evidence	Checks record 8% free space, inode usage, largest paths, oversized logs, and container usage.	Policy permits only bounded collection from declared sources.	Freshness, completeness, truncation, and service identity are checked.
Diagnosis	A hypothesis links growth to logrotate policy and a recent deploy; evidence identifying the owner of the retention rule is still missing. A bounded follow-up Evidence request confirms that owner before planning continues.	Missing authority produces escalation rather than an invented owner.	Guardian checks supporting and contradicting evidence, falsification checks, and the new owner evidence.
Plan	The planner proposes temporary volume expansion followed by correction of the retention rule, assesses blast radius, and prepares verification and rollback or compensation plans.	R1 autonomy, C2 capability, and impact I3 permit a human-reviewed proposal only; destructive cleanup is denied.	A gate checks preconditions, reversibility, impact, and Intent alignment.
Approval	HumanDecision binds approval of both ordered actions to scope, expiry, and the exact Plan version.	The now-confirmed authorized owner approves or denies the change; self-approval is impossible.	Signature, validity, authority, separation of duties, and unchanged inputs are checked.
Controlled Change	A separate Executor applies only the approved volume expansion and retention-rule correction, in that order, with scoped credentials and writes an ExecutionRecord with the case-scoped attempt budget and index.	Governance Mesh rechecks policy and produces an admission receipt; the circuit breaker blocks execution when the budget or cooldown rule is violated.	Exact target, package digest, idempotency or compensation basis, and actual actions are checked; the outcome receipt binds to the same attempt.
Verification	After a 24-hour observation window, evidence shows 24% free space, correct log rotation, no business-data deletion, and no SLO degradation. Outcome Check records `succeeded` and binds the facts to the exact change and evidence digests.	Policy requires every criterion and prohibits closure on `unknown`.	Independent evaluation confirms the after-state, observation window, criterion completeness, and residual risks.
Learning	Outcome, decision, and evidence digests are linked; runbook and policy updates are proposed.	Each policy or runbook change enters a new governed cycle.	Trace proves that Learning uses a verified Outcome and does not rewrite history.

AI explains and plans. Governance constrains. A human approves. An executor applies. Outcome proves the result.

23.1. A Small Team and Its Service Owner (Informative)

Informative example — a small team and its service owner. An engineer combines development and on-call work; another suitably authorized, competent participant provides the required independent review. That participant may be external. The service owner sets the priority and acceptable bounds and names the person who communicates with users. These functions need not be separate departments, but one person changing role names does not create independence.

The first Intent may ask only for a verified disk-growth report. In that case, a supported report can satisfy the Intent without a target change. If the Intent instead asks for service restoration and a proposed change is denied, the trace records the refusal and the result against the original criteria; it does not invent an ExecutionRecord or record restoration as successful. If required independent review is unavailable, the team continues only permitted evidence collection, analysis, preparation, and escalation.

After the case, the owner receives the verified result, residual risks, and proposed reliability or development work. A runbook edit remains a proposal until its applicable review and admission process completes. When development later changes the product, the operational acceptance criteria can be checked again using new evidence linked to the earlier case.

This is a teaching example, not evidence of a runtime implementation. The limited C4 profile in Appendix G covers an isolated, resettable, non-production test domain; neither this example nor passing the offline validators extends that domain.

24. Methodology Adoption Success Criteria

Success is evaluated by comparing like-for-like periods before Agent-Ops adoption and after adoption and stabilization. The primary measure is the number of completed units of work: requests and incidents brought to an explicit terminal result that is accepted by the accountable owner and supported by the evidence required for that case. Both owner-accepted `succeeded` and `failed` terminal results may count as completed work, but their outcome classes are reported separately; `unknown` or incomplete cases do not count as completed. Report both the absolute count and units completed per unit of time. Show incoming volume, backlog change, and team effort alongside it so that clearing accumulated backlog or adding resources is not misrepresented as a methodology improvement.

Before comparison, fix equal window lengths, service boundaries, data sources, inclusion and exclusion rules, severity and impact segments, and available team capacity in labor hours. Take the baseline before adoption and the comparison window after an agreed stabilization period. If service scope, team composition, or workload changes materially, report those populations separately rather than combining them into one value.

Measure	Before-and-after comparison
Completed units of work — primary measure	Count requests and incidents with an accepted terminal result per window and per unit of time. Also report incoming units, backlog change, labor hours, and the severity and service distribution.
Triage time	Duration from the request registration timestamp in the system of record to the first accepted diagnostic conclusion or governed next-step decision. Compare median and p90 by severity and service; show waiting-on-customer time separately.
Evidence completeness	Evidence elements required for the case that have authoritative, complete, current evidence divided by all evidence elements required for that case. Exclude only proven non-applicable elements; `unknown`, stale, conflicting, and missing elements remain in the denominator, are reported separately, and never enter the numerator.
Rework	Reopened or rejected diagnoses and plans, plus correction cycles, divided by accepted cases; also compare mean resubmissions per case.
Safe decisions	Decisions that passed applicable policy and authority gates without unauthorized action divided by all decisions in scope. Report denials, escalations, policy violations, incidents, and rollbacks separately; absence of a recorded incident is not proof of safety.

Measure	Before-and-after comparison
Verified outcomes	Outcome Checks with exact-target after-state evidence and an explicit `succeeded`, `failed`, or `unknown` result divided by all approved intents whose Verification was due in the comparison window. An approved intent that did not reach Verification remains in the denominator and is reported as missing/`unknown`; only `succeeded` counts as achieved.
Promotion readiness	For each exact action class, environment, impact band, and agent/model/toolchain version, report shadow cases with complete engineer decisions, actual actions, and verified Outcomes divided by all eligible shadow cases. Also report safe agreements, unsafe agreements, disagreements by class, policy violations, overrides, and `unknown` Outcomes as absolute numerators and denominators. No threshold promotes a capability automatically.

Higher completed-unit volume counts as successful methodology adoption only when evidence completeness and decision safety are maintained or improved. Publish before-and-after values with their absolute numerators and denominators, not percentages alone.

25. Phased Adoption

This table is an adoption roadmap, not the transition order of an individual case. Each case still establishes its Intent, scope, authority, and required evidence before the relevant work, as specified in Sections 6–7. Later roadmap entries do not defer those prerequisites.

Stage	Outcome
0. Boundaries + governance	Production scopes, data classification, authority, denylist, and foundational Policy Hooks.
1. Evidence foundation	Audit-fast/regular, schemas, redaction, and exact-commit validation.
2. Declared / observed	IaC sources, effective policy, and classified drift.
3. Project Harness	Owners, SLOs, policies, runbooks, permissions, stable IDs, and policy registry.
4. Observability readiness	Telemetry coverage, freshness, alerts, and diagnostic references.
5. Intent + Discovery	Outcome Hypothesis, HITL Decision Map, and validation plan.
6. Evidence Bundle + evals	Versioned decision trace, four eval classes, and Outcome Check.
7. Incident/RCA	Timeline, hypotheses, contradictions, and Discovery loop.
8. Planner + Guardian	Human-reviewed proposal, independent evaluation, and safe handoff.
9. Shadow operations	Real-case investigation and planning with `execution_authority: none`; compare the agent proposal with the engineer decision, actual action, and verified Outcome by action class.
10. Gated apply	Separate executor for explicitly promoted and approved bounded actions only.
11. Bounded self-healing	Proven repeatable scenarios only; disabled by default.

> *Pilot -> standardize -> scale. Every stage closes with evidence and does not inherit the next stage's authority.*

A team can obtain sustained value from audit, diagnosis, and change preparation without progressing to execution. Progress is assessed by verified outcomes, maintained controls, and the team's total effort, not by autonomy alone. Each additional capability remains subject to its applicable evaluation and admission requirements. C5 remains a future, disabled-by-default mode; every real target change under the current operating boundary retains explicit human approval and a separate human start of the deterministic Executor.

26. Expected Benefits and Positioning

Agent-Ops should be positioned as an AI-native Server Operations Methodology and Operations-as-Code framework intended to support safe operations and technical support of server, cloud, and Kubernetes infrastructure.

The following benefits are expected, not demonstrated comparative results:

- less dependence on informal knowledge held by individual engineers;
- repeatable triage and a common standard across projects;
- faster discovery of missing evidence and observability gaps;
- safer LLM use in production operations;
- verifiable change plans instead of free-form recommendations;
- linked audit, incident, change, and postmortem artifacts;
- managed-service scaling without universal agent privileges.

No percentage improvement is claimed without measurements. Future research claims depend on comparison with a strong composition of existing mechanisms under matched assumptions. If no advantage is established, the contribution is described as an operational profile, reference implementation, or measurement methodology, according to the artifacts actually delivered. Any observed advantage is limited to the evaluated environments and fault classes.

> *Agent-Ops: Intent defines the result; evidence proves reality; Governance constrains; a human approves; an executor applies; Outcome verifies.*

27. Bilingual Normative Publication

For documents at the same source revision, if the English and Russian versions differ in meaning, the English normative member prevails. Every cataloged English document has exactly one Russian member at that revision. Structural parity is machine-checked; semantic equivalence requires an owner-designated external bilingual reviewer.

A translation MAY clarify a term in Russian, but it MUST preserve public identifiers, schema values, status values, normative force, and contract meaning. In Russian prose, the reader-facing Russian name comes first and the English term MAY follow in parentheses at its first introduction or when a standalone section needs the reminder. Literal type and field names, status values, registered product names, and source titles remain exact. Approved exceptions identify the exact pair, revision, immutable target, differing structure, and reviewer.

The closed publication set consists of this white paper, the standards map, and the controlled glossary. The pair and publication manifests identify exact members and distinguish candidate, reviewed, generated, and pending artifacts.

28. Status of This Edition

Edition v0.5.2, source revision `aom-04-r9`, supersedes edition v0.5.1 and, like it, publishes two independently versioned profiles: limited C4 Human-approved Apply and Evidence Acquisition. Their stable identities, scope, explicit non-goals, assumption and claim boundaries, trusted computing bases, common-cause boundaries, separate bounded executable formal state-transition models, independently versioned relation catalogs, and small semantic interface are normative candidate content. This edition also publishes bounded implementation slices: six C4 record schemas and `agent-ops.relation-validation@1.0.0`, five Evidence Acquisition schemas and `agent-ops.evidence-relation-validation@1.0.0`, and their composed `agent-ops.integrated-conformance@1.0.0` checker with a frozen pre-runtime evaluation protocol. Positive, schema-negative, relation-negative, and paired cross-boundary fixtures remain synthetic and offline. The edition does not implement a runtime, provide a complete lifecycle validator, report a runtime evaluation, or establish production safety. The agent identity, session baseline and session stop, aggregate bound, and session-scoped reasoning-runtime requirements added in v0.5.1 (§3.3, §11.2, §16.2, §19.1, §19.3, §20.1, §20.3) are `target` and add no implementation slice; Appendix G and the formal models are unchanged. This edition clarifies case boundaries, organizational responsibilities, completion without change, and adoption without mandatory autonomy growth; it adds informative small-team, lost-response continuation, and full-adoption-cost explanations. These clarifications and examples change no profile, schema, relation catalog, semantic interface, formal model, execution authority, or completion-evidence requirement. Figure edition labels are aligned with v0.5.2. The paired v0.5.2 PDFs render this source revision and have deterministic Build/Test evidence, but neither those artifacts nor earlier reviews approve the successor. Release still requires fresh exact-target owner and bilingual review, complete release and reproducibility evidence, and separate authorization for the protected release environment. Repository presence alone does not make the edition released.

Appendix A. 47 Baseline Checks

Domain	check_id
Inventory	inventory.host.identity
Inventory	inventory.os.version
Inventory	inventory.kernel.version
Inventory	inventory.cpu.summary
Inventory	inventory.memory.total
Inventory	inventory.disk.layout
Inventory	inventory.time.sync
Health	health.cpu.load_average
Health	health.cpu.iowait
Health	health.memory.pressure
Health	health.memory.swap_usage
Health	health.memory.oom_events
Health	health.systemd.failed_units
Services	services.systemd.active_inventory
Services	services.systemd.enabled_inventory
Services	services.systemd.restart_count
Services	services.processes.orphan_long_running
Services	services.containers.inventory
Network	network.ports.listening

Domain	check_id
Network	network.ports.bind_address
Network	network.firewall.status
Network	network.dns.resolution
Network	network.tls.certificate_expiry
Logs / Rotation	logs.journald.disk_usage
Logs / Rotation	logs.journald.retention_policy
Logs / Rotation	logs.logrotate.configuration
Logs / Rotation	logs.files.oversized
Logs / Rotation	logs.docker.rotation
Logs / Rotation	logs.system.high_severity_errors
Disk / Cleanup	disk.filesystem.free_percent
Disk / Cleanup	disk.filesystem.inode_usage
Disk / Cleanup	disk.usage.largest_directories
Disk / Cleanup	disk.usage.large_files
Disk / Cleanup	disk.cleanup.package_cache
Disk / Cleanup	disk.cleanup.docker_usage
Backups / Restore	backups.jobs.presence
Backups / Restore	backups.last_success.age
Backups / Restore	backups.coverage.critical_data
Backups / Restore	backups.retention.policy
Backups / Restore	backups.integrity.last_check
Backups / Restore	backups.restore.last_test
Monitoring readiness	monitoring.node_exporter.presence
Monitoring readiness	monitoring.textfile_collector.configured
Monitoring readiness	monitoring.audit_metrics.freshness
Monitoring readiness	monitoring.prometheus.scrape_visibility
Monitoring readiness	monitoring.alerts.baseline_rules
Monitoring readiness	monitoring.logs.collection_readiness

Appendix B is intentionally maintained as the standalone [Agent-Ops glossary](../normative/glossary.en.md) so the controlled vocabulary can be versioned and reviewed as its own bilingual pair. The sequence therefore continues from Appendix A to Appendix C in this white paper.

Appendix C. Source Materials

- AI-Disrupt PDLC, complete practical guide, version 2.0, Sber, June 2026; methodology author: Kirill Menshov. The Section 4.3 table is the source of R0-R5; Section 4.5 supplies the T1-T5 notation and sequence, adapted to the Agent-Ops independent-control boundary in Section 5. Source T5 and the separate Agent-Ops Executor are not equivalent. Section 2.11 supplies the completion Evidence Bundle pattern adapted in Section 14.3; Section 2.13 supplies the Governance Mesh antecedent for Section 18. Its context-management and Context Supply Chain patterns also inform Section 20. These are operational adaptations, not claims of invention or adoption of the source's product-development lifecycle: <https://aipdlc.ru/documents/en/whitepaper_full_en.pdf>.
- OWASP Agentic Skills Top 10 (AST01-AST10), version 1.0, 2026 — the source of the edition 0.3.2 requirements on the skill supply chain, audit receipts, and coverage records: <https://owasp.org/www-project-agentic-skills-top-10/>.
- Ganesh Gurudu, "Building an AI Agent That Runs Your SRE Operations — What I Learned, What Works, and How You Can Do It Too," 8 April 2026 — an informative implementation-experience source for shadow evaluation, attempt budgets, agent-runtime observability, context assembly, and integration admission; it creates no dependency on the article's architecture, risk scale, deployment topology, or technology stack: <https://blog.stackademic.com/building-an-ai-agent-that-runs-your-sre-operations-what-i-learned-what-works-and-how-you-can-do-8a3801124bdc>.
- Ali Shazal and Matthew Koen, Anthropic, "A guide to the anatomy of effective commerce agents," 2 September 2026 — an informative source outside the operations domain for proposal-versus-approval boundaries, authoritative identifiers, serialized writes and resulting-state limits, deterministic context demarcation, and constructed-state evaluation; it creates no dependency on the article's commerce architecture, model selection, latency techniques, or deployment stack: <https://claude.com/blog/the-anatomy-of-effective-commerce-agents>.
- NIST SP 800-207 *Zero Trust Architecture*, August 2020 — prior source of the principles of explicit verification, least privilege and assumed breach referenced in §3.3: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf>.
- Alexander Klizhentas and Diana Jovin, Teleport, *From Zero Trust to Agent Trust*, 2 July 2026 — an informative source outside the methodology for agent identity attestable to a grantor, delegation, ephemeral runtimes and bounded collective autonomy. It creates no dependency on the product, identity platform or deployment topology of the paper: <https://goteleport.com/resources/white-papers/zero-trust-to-agent-trust/>.
- External standards, regulations, and their conformance mapping are maintained in the standalone [Agent-Ops standards map](../normative/standards-map.en.md).

Appendix D. Normative Result Contract

Every check produces an independent schema-backed result. Summary, Prometheus exposition, and AI-facing projections are derived views and may not change its semantics.

Field	Required Semantics
check_id	Stable identifier of the logical check.
role / technology	Applicability to role, platform, and dialect pack.
status	Exactly one of ok, info, warning, critical, unknown, not_applicable, or error.
severity	Normative S0-S5 scale; not autonomy, impact, or maturity.
applicable	Explicit applicability; a missing provider does not automatically mean not_applicable.
evidence_refs	Verifiable references with source, timestamp, freshness, authority, and digest. A reference proves neither that its acquisition was valid nor that it is sufficient for the consuming claim unless the applicable evidence requirement is evaluated.
policy_source	Applied rule, version, owner, and digest.

Field	Required Semantics
recommendation	Plan-only proposal; never approval or execution authority.
completeness / truncation	Completeness, physical limits, dropped items, and compaction reason.
schema / tool identity	Schema, tool, profile, and canonical-writer versions.

Appendix E. Check Result Status Semantics

Axis	Value	Interpretation
status	ok	The applicable check completed successfully, produced sufficient evidence, and satisfied all policy requirements.
status	info	Neutral fact that is not a violation.
status	warning	Confirmed deviation requiring attention.
status	critical	Confirmed critical deviation.
status	unknown	Evidence is insufficient, stale, incomplete, or contradictory.
status	not_applicable	The check is proven irrelevant to the normalized role.
status	error	Technical collection, parsing, evaluation, or writer failure.

Unknown is not OK; error is not unknown; not_applicable requires proven non-applicability.

Appendix F. Taxonomy and Namespace Registry

A scale's namespace is a closed set of values with its own meaning. Full definitions of every level and their application are in Sections 5.1–5.7; this registry lists every value and links to the corresponding definition.

Namespace	All values	Definition and application
M0-M6	M0 unmanaged / unknown; M1 initial audit; M2 regular diagnostics; M3 observability; M4 managed operations; M5 approval-controlled semi-automated recovery; M6 bounded self-healing / AI-assisted SRE, a future mode.	[Process maturity, Section 5.1](#51-process-maturity-m0-m6). Never grants authority.
S0-S5	S0 informational; S1 low; S2 moderate; S3 material; S4 high; S5 critical.	[Check result severity, Section 5.2](#52-check-result-severity-s0-s5). Includes uncertainty risk; neither replaces status nor proves a deviation.
C0-C5	C0 observe; C1 explain; C2 plan; C3 prepare change; C4 human-approved apply; C5 bounded self-healing, a future capability disabled by default.	[Capability, Section 5.3](#53-capability-c0-c5); execution boundary in Section 18.2. Capability does not create authority.
I0-I5	I0 no production impact, read-only / offline; I1 isolated reversible non-production; I2 localized reversible low-criticality; I3 production service-local; I4 multi-service, critical, or sensitive; I5 systemic or irreversible impact, data loss, or regulatory consequences.	[Operational impact, Section 5.4](#54-operational-impact-i0-i5): each level's boundaries and controls; admission in Section 18.
R0-R5	R0 read-only; R1 proposals only; R2 feature branch with merge-request review; R3 auto-merge after passing evals; R4 multi-session work with checkpoints; R5 full autonomy in an isolated sandbox.	[Autonomy, Section 5.5](#55-autonomy-r0-r5); policy application in Section 18. Source: the Section 4.3 table in AI-Disrupt PDLC 2.0, cited in Section 5.

Namespace	All values	Definition and application
T1-T5	T1 review; T2 verification; T3 redirect; T4 block; T5 arrange remediation through a separately authorized path.	[Guardian maturity, Section 5.6](#56-guardian-maturity-t1-t5). Adapted from Section 4.5 of AI-Disrupt PDLC 2.0; the Guardian receives no authority to change the target.

[The joint application example in Section 5.7](#57-applying-the-scales-to-one-case) explains the six different assessment subjects. Equal numbers on different scales are not interchangeable.

External sources and conformance mappings formerly carried in Appendix J are maintained in the standalone [Agent-Ops standards map](../normative/standards-map.en.md).

Appendix G. Limited C4 Human-approved Apply Profile 1.0.0

G.1. Status, identity, and scope

The stable public identifier of this profile is `agent-ops.c4-human-approved-apply@1.0.0`. This appendix is the normative contract for its boundary, assumptions, claim vocabulary, and trusted computing base and, together with the versioned artifacts in `formal/`, defines the bounded executable candidate. The six C4 schemas and public relation validator implement only the named offline cross-record slice. An implementation MUST NOT claim runtime conformance to this profile from this appendix, the bounded formal state-transition model, schema-valid records, or passing fixtures alone.

The profile covers one human-approved application of one canonical `ChangePackage` by a separately human-started deterministic Gated Executor. The package binds one Intent and canonical Outcome identity, exact target UID, typed operation, canonical arguments, decision-critical dependencies, validity interval, policy and approval epochs, and attempt budget. Planner and Guardian have no target-write credentials. The trusted gateway or cooperating backend adapter checks authority at use and admits or rejects the operation at a declared linearization point. An authoritative durable log preserves lifecycle events; a read-only effect observer classifies actual target changes and observation completeness independently of the Executor.

The initial reference domain contains only predeclared typed operations against an isolated, resettable, non-production test target. Every operation declares target identity, version semantics, admission semantics, idempotency or reconciliation behavior, conflict domain, fencing support, observation coverage, and Outcome oracle. A deployment MAY implement a narrower subset, but MUST NOT silently broaden it.

The following are outside profile 1.0.0:

- standing C5 authority or approval created automatically for a human;
- arbitrary shell commands, unconstrained scripts, or undeclared third-party APIs;
- production or irreversible effects, and general isolation from non-cooperating writers;
- exactly-once claims for an arbitrary external effect;
- recovery claims while the required control plane, observer, or Outcome oracle remains unavailable;
- claims that schema validation alone establishes cross-record authority, temporal safety, implementation safety, scientific novelty, or production readiness.

G.2. Guarantee vocabulary and claim rules

The following terms are not interchangeable:

Term	Controlled meaning
admission safety	No operation is admitted at the declared enforcement point unless the profile's authority predicate holds for that exact package, target, attempt, policy epoch, approval epoch, and use time.
effect knowledge	The classified knowledge `applied`, `not_applied`, `partial`, or `unknown` about target changes attributable to an attempt, accompanied by observer coverage. It is not the attempt state or Outcome.

Term	Controlled meaning
Outcome success	The canonical Outcome oracle reports `succeeded` against the original Intent and bound after-state evidence. An admitted or applied operation need not succeed.
recovery	Every admitted unresolved obligation eventually reaches an authorized terminal disposition and required service or test-state restoration under stated availability and reversibility assumptions. A safe stop is not recovery.
retry safety	A repeated delivery cannot create an effect outside the operation's declared semantics because the same backend/idempotency key is enforced, complete observation proves no prior effect, or authorized reconciliation determines the next action. Compensation alone is not retry safety.
conflict safety	Cooperating writers that overlap any target write or decision-critical read are serialized or rejected within the declared enforcement domain, and stale writers are fenced.
continuity	Attempts, reservations, fences, approval and policy epochs, and unresolved effects survive checkpoint, handoff, restart, and lease expiry without reset or omission.
evidence completeness	The observer explicitly records what it covered, excluded, or could not determine. No observed change under incomplete coverage does not mean `not_applied`.

Safety and liveness claims MUST be stated separately. A claim MUST name its profile version, assumptions, trusted components, enforcement domain, exact operation family and target class, observation coverage, and validation evidence. Removing or failing an assumption narrows or invalidates the claim; it MUST NOT be hidden by an aggregate success rate. `unknown`, partial effects, timeouts, and incomplete observation remain visible.

This appendix establishes the vocabulary and assumptions below. G.7 supplies the bounded executable definitions; they become claimable runtime guarantees only after the exact candidate and its conformance obligations are accepted and the exact implementation supplies current evidence.

G.3. Assumption catalog

Each assumption has a stable identifier. Implementations MUST declare `satisfied`, `not_satisfied`, or `unknown` with evidence for every applicable assumption; only `satisfied` supports a dependent claim.

ID	Assumption and required evidence	Claim consequence when removed, failed, or unknown
C4-A01	The target is isolated, resettable, non-production, and admits only the registered typed operation set. Evidence identifies the target class and reset procedure.	Profile applicability and recovery evaluation are not established.
C4-A02	Every writer that can overlap a protected write or decision-critical read cooperates with the declared gateway, reservation, and fencing domain. Out-of-domain writers are enumerated and observed.	Admission and conflict safety apply only inside the cooperating domain; an outside writer can invalidate them.
C4-A03	The gateway or backend adapter has a documented atomic admission point that checks the bound authority predicate before accepting the mutation.	A pre-call check provides local policy evidence only; no atomic remote-authorization claim is permitted.
C4-A04	The backend supplies immutable target UIDs and monotonic or otherwise ABA-resistant decision-critical resource versions.	Name reuse or ABA can redirect an approval or make stale evidence appear current.
C4-A05	Each operation declares backend idempotency, same-key replay, partial-effect, cancellation, reconciliation, and fencing semantics.	Retry, cancellation, and stale-writer safety cannot be inferred.
C4-A06	The authoritative lifecycle log is append-only, durably ordered, and atomic for each accepted event; corruption and unavailability are detectable.	Event absence, order, attempt consumption, and unresolved obligations cannot be trusted.

ID	Assumption and required evidence	Claim consequence when removed, failed, or unknown
C4-A07	The reservation store provides durable atomic acquisition/update and monotonic fencing tokens over the declared conflict domain.	Concurrent or expired writers can both act; conflict safety and continuity fail.
C4-A08	Principal identity, signer identity, policy identity, and key lifecycle are authenticated; signing keys are protected and revocation is visible at use.	Package or approval provenance and authority validity are not established.
C4-A09	Planner, Guardian, observer, and ordinary users lack target-write credentials; Executor credentials are operation-, target-, time-, and attempt-scoped.	A bypass path can create an unattributed or unauthorized effect.
C4-A10	Time sources, validity-bound interpretation, and maximum clock uncertainty are declared. Admission treats the earliest possible expiry and latest possible `not_before` conservatively.	An apparently valid proof can be expired or not yet valid at the backend admission point.
C4-A11	The read-only effect observer declares resource, field, time, and external-effect coverage and reports `complete`, `partial`, or `unknown`. It is operationally separate from the Executor's response path.	Absence of an observed change cannot establish `not_applied`; effect knowledge remains `unknown`.
C4-A12	The canonical Outcome oracle is bound to the original Intent, target UID, after-state evidence, and completion rules; its independence from the operation path is declared.	Effect knowledge cannot be promoted to Outcome success, and service recovery cannot be claimed.
C4-A13	External controllers, asynchronous reconcilers, admission webhooks, and backend side effects are enumerated with their writer and observation domains.	Attribution, conflict classification, and resulting-state claims exclude undeclared controllers.
C4-A14	Availability assumptions are explicit for the identity/policy service, log, reservation store, gateway/backend, observer, and Outcome oracle. Loss of a required service causes fail-closed admission where enforceable.	Safety may degrade to a stop; eventual completion, reconciliation, and recovery are not guaranteed during unavailability.
C4-A15	Irreversible effects are excluded. Every admitted operation has a declared reset or separately authorized compensation/reconciliation path whose residual effects are observable.	Restoration and bounded residual-risk claims are invalid.
C4-A16	Credential, signing-key, gateway/backend, log, reservation-store, observer, or oracle compromise is outside the corresponding component's guarantee and is separately detected and handled.	A compromised TCB component can forge or suppress the fact it is trusted to enforce or observe.
C4-A17	Checkpoints and summaries are derived from, and can only refine, authoritative durable log and reservation state. Resume reads the authoritative state before proceeding.	Restart or handoff can reset attempts, epochs, reservations, fences, or unresolved obligations.
C4-A18	Canonicalization and digest algorithms are versioned, deterministic, collision-resistant for the profile, and identically interpreted by approval and admission components.	Different packages can share or appear to share an approval identity.
C4-A19	The approving human and starting human are authenticated, are shown the canonical package identity and declared consequences, and act under an applicable organizational authority rule.	A valid signature or UI event does not establish accountable human authority for the operation.

G.4. Trusted computing base

Planner and model output are untrusted proposal inputs and are not in the admission TCB. Guardian is not trusted to create authority; it is trusted only for the exact independent checks whose inputs and paths satisfy the table. Every implementation MUST publish an equivalent deployment-specific table.

Component	Enforcement or observation boundary	Failure or compromise assumption	Required independence	Limit on claims
Identity and policy authority	Authenticates principals and supplies current organizational authority, policy epoch, revocation, and scope.	Availability is declared; forged identity, stale epoch, or compromise invalidates authority claims.	Separate from Planner and target-write path; revocation checked at use.	Cannot prove package integrity, backend admission, effect, or Outcome.
Signer and key store	Signs the exact canonical human decision or attestation and protects signing material.	Key extraction, unauthorized signing, or hidden revocation invalidates provenance.	Approval key is unavailable to Planner, Guardian, and Executor.	Signature proves origin and integrity, not safety or backend use.
Canonicalizer and digest implementation	Produces the byte identity approved and later checked at admission.	Version skew, ambiguity, or collision invalidates package binding.	Approval and admission may use separate instances but MUST use the same accepted algorithm/version and test vectors.	Cannot establish authority, freshness, or effect.
Authoritative durable log	Orders immutable lifecycle facts, attempts, admissions, effect classifications, receipts, reservations, reconciliation, compensation, and checkpoint lineage.	Accepted-event atomicity and durability are assumed; detectable unavailability fails closed where required.	Writer identities are authenticated; observer and Executor cannot rewrite history.	Logged assertion is not proof of an external effect unless backed by the declared observer.
Reservation and fence store	Serializes declared conflict domains and issues monotonic fence tokens.	Atomic durable update and backend fence enforcement are assumed.	Executor cannot reset epochs or mint an accepted fence outside policy.	Protects only cooperating writers and declared read/write conflicts.
Gateway or cooperating backend adapter	Checks authority at use and admits/rejects the typed operation at the declared linearization point; enforces version preconditions, key, and fence.	Its admission semantics are trusted; compromise or a bypass writer invalidates admission safety.	No Planner/Guardian credential; policy and package identities are independently verifiable.	A separate pre-call check cannot substitute for this boundary.
Effect observer	Reads target and declared external surfaces, attributes changes, and reports coverage and `applied` / `not_applied` / `partial` / `unknown`.	Coverage gaps and unavailability are explicit; compromise can forge effect knowledge.	Read-only credentials and an observation path independent of the Executor response.	Cannot grant authority or by itself decide Outcome.
Workload and Outcome oracle	Evaluates the original Intent against canonical after-state and service/test evidence.	Oracle correctness and declared coverage are assumed; unavailable or conflicting evidence yields `unknown`.	Separate from Planner and mutation response; common data sources are disclosed.	Outcome success does not prove admission safety or absence of residual effects.
Human approver and human starter	Supplies accountable approval and a separate start event for the exact package.	Authenticated identity, comprehensible canonical display, and organizational authority are assumed.	Cannot be replaced by an agent, metric, or Guardian result; applicable self-approval rules still apply.	Human action cannot repair stale evidence, package mismatch, backend bypass, or absent observation.

Component	Enforcement or observation boundary	Failure or compromise assumption	Required independence	Limit on claims
Time and version sources	Supply bounded time uncertainty, target UID, policy/approval epochs, and decision-critical versions used at admission.	Bounds, monotonicity or ABA resistance, and availability are declared.	Sources are authoritative for their named facts; shared sources are common-cause dependencies.	TTL alone never proves decision-relative freshness.

G.5. Common-cause failures and Guardian independence

The deployment record MUST enumerate shared language models, prompts, evidence producers, credentials, keys, canonicalizers, clocks, version services, policy and identity services, logs, storage systems, network paths, target APIs, observers, and Outcome data sources. Two checks that depend on the same failing fact are not independent evidence for that fact. Diversity of language-model name, prompt wording, process, or host MAY improve detection but does not establish independence unless authority, credentials, sources, and observation paths are also separated for the claimed failure class.

Guardian MAY block, redirect, or request separately authorized remediation. It MUST NOT approve on behalf of a human, mint or widen an Executor credential, modify the target, rewrite the authoritative record, conceal `unknown`, or repair the evaluated artifact outside the logged lifecycle. A common false proof supplied to both Planner and Guardian is retained as an executable counterexample and remains a required negative conformance fixture.

G.6. Claim matrix

Claim	Kind	Required assumptions and trusted components	Enforcement domain and present status
C4-G01 Exact-package admission safety	safety	C4-A02-A04, C4-A08-A10, C4-A18-A19; identity/policy authority, signer, canonicalizer, gateway/backend, time/version sources	Cooperating gateway/backend and exact typed operation only; bounded by exact package binding, current approval, current human start, held reservation, current fence, and authorized admission (`C4-R01`-`C4-R06`) plus P1/P2/P6; not yet a runtime claim.
C4-G02 Conflict and stale-writer safety	safety	C4-A02, C4-A04-A07, C4-A13; reservation/fence store and cooperating backend	Declared conflict domain and cooperating writers only; bounded by held reservation (`C4-R04`), current fence (`C4-R05`), and P4.
C4-G03 Retry safety	safety	C4-A05-A07, C4-A11, C4-A15; backend, log, observer, and authorized reconciler	Registered operation family only; safe retry (`C4-R08`) requires a backend key, complete negative Evidence, or authorized reconciliation; compensation alone is insufficient.
C4-G04 Authority and budget continuity	safety	C4-A06-A10, C4-A17-A18; log, reservation store, identity/policy, canonicalizer, time/version sources	Checkpoint, handoff, restart, and lease-expiry transitions; bounded by a monotonic attempt budget (`C4-R07`), checkpoint refinement (`C4-R12`), and P3.
C4-G05 Honest effect knowledge	epistemic safety	C4-A06, C4-A11, C4-A13; log and effect observer	Declared observation surface only; supported effect knowledge (`C4-R09`) and the applicability, freshness, coverage, disposition, sufficiency, provenance, and common-cause interface relations (`CEI-R01`-`CEI-R07`) require incomplete coverage to remain `unknown`; bounded by P5.

Claim	Kind	Required assumptions and trusted components	Enforcement domain and present status
C4-G06 Outcome classification integrity	epistemic safety	C4-A06, C4-A12-A13; log and Outcome oracle	Original Intent and declared oracle coverage only; bounded by canonical Outcome classification (`C4-R10`) and P7.
C4-G07 Eventual obligation resolution	liveness	C4-A01, C4-A05-A07, C4-A11-A15, C4-A17; all required control-plane services and human escalation path	Resolved obligation (`C4-R11`), allowed terminal release (`C4-R13`), and L1 check an authorized bounded resolution path under availability and reversibility; unbounded liveness is not proved.

G.7. Executable formal state-transition models and relation catalog

The executable specification defines two compositional, versioned formal state-transition models: the C4 authority-and-effect model in `formal/c4.go` and the Evidence Acquisition assertion-and-sufficiency model in `formal/evidence.go`. These are deterministic specifications explored by the checker, not language models. `agent-ops.c4-evidence-interface@1.0.0` connects them without merging their authority domains. The independently versioned C4 catalog is `agent-ops.c4-relations@1.0.0`; every relation names its authority, check time, and failure disposition. The C4 state includes canonical package digest and version, target UID and decision-critical versions, approval and policy epochs, bounded time interval, human approval and start events, attempt budget and consumed attempts, conflict domain, reservation and fence, admission state, effect knowledge, Outcome state, reconciliation or compensation obligations, and checkpoint lineage.

Relation identifiers are precise normative locators, not reader-facing explanations. On first use, prose gives the readable name and meaning before the identifier; later compact references may use the identifier alone. The following index covers all three relation catalogs.

ID	Reader-facing and catalog name	Meaning
`C4-R01`	Exact package binding (`package_bound`)	Binds approval and admission to one canonical package, exact target, and decision-critical versions.
`C4-R02`	Current approval (`approval_current`)	Requires the signed human decision, identity, policy, and revocation state to be current at admission.
`C4-R03`	Current human start (`human_start_current`)	Requires an authenticated human-start event before the first attempt is admitted.
`C4-R04`	Held reservation (`reservation_held`)	Requires the authoritative reservation for the declared conflict domain at reservation and admission.
`C4-R05`	Current fence (`fence_current`)	Requires the cooperating backend to reject a stale writer at mutation.
`C4-R06`	Authorized admission (`admission_authorized`)	Requires the cooperating gateway or backend adapter to authorize the operation at the declared atomic admission point.
`C4-R07`	Monotonic attempt budget (`attempt_budget_monotonic`)	Prevents checkpoint or restart from resetting consumed attempts in the durable lifecycle log.
`C4-R08`	Safe retry (`retry_safe`)	Allows retry only through declared same-key semantics, complete applicable negative Evidence, or authorized reconciliation.
`C4-R09`	Supported effect knowledge (`effect_knowledge_supported`)	Requires effect classification to be supported by the declared observer's Evidence; otherwise the effect remains `unknown`.
`C4-R10`	Classified Outcome (`outcome_classified`)	Requires the canonical Outcome oracle and applicable Evidence; otherwise Outcome remains `unknown`.
`C4-R11`	Resolved obligation (`obligation_resolved`)	Requires an admitted unresolved effect to reach an authorized reconciliation or compensation disposition.

ID	Reader-facing and catalog name	Meaning
C4-R12	Checkpoint refinement (checkpoint_refines)	Allows a checkpoint only to refine, never reset or omit, authoritative log and reservation state.
C4-R13	Allowed terminal release (terminal_release_allowed)	Releases a case only when its authoritative lifecycle state permits it.

ID	Reader-facing and catalog name	Meaning
EVD-R01	Applicable requirement (requirement_applicable)	Establishes whether a named consumer requirement applies to collection and use.
EVD-R02	Authorized collection profile (collection_profile_authorized)	Requires an authorized versioned collection profile before each attempt.
EVD-R03	Append-only attempts (attempt_append_only)	Preserves every attempt and failure through retry, checkpoint, and restart.
EVD-R04	Bound receipt (receipt_bound)	Binds a collection receipt to the exact target, source, and receipt event.
EVD-R05	Artifact integrity (artifact_integrity)	Verifies canonical bytes and digest before artifact admission or consumer use.
EVD-R06	Complete derivation lineage (derivation_lineage_complete)	Preserves provenance through every transformation that produces an artifact or assertion.
EVD-R07	Assertion disposition (assertion_disposition)	Records whether the bounded evaluator supports, refutes, does not test, or cannot determine an assertion.
EVD-R08	Valid negative observation (negative_observation_valid)	Accepts a negative assertion only with complete applicable coverage and detection capability.
EVD-R09	Fresh at use (fresh_at_use)	Evaluates exact target/source identity, time, and critical versions at the consumer decision.
EVD-R10	Supported independence (independence_supported)	Treats evidence as independent only when declared common-cause dependencies support that claim.
EVD-R11	Computed completion (completion_computed)	Computes bundle completion from the consumer requirement and complete acquisition lineage.
EVD-R12	Sealed bundle (bundle_sealed)	Permits sealing only over the computed completion state.
EVD-R13	Consumer-specific sufficiency (consumer_sufficient)	Evaluates sufficiency against the named consumer rule version at its decision.
EVD-R14	Propagated invalidation (invalidation_propagated)	Makes dependent facts unknown after a relevant target, source, profile, or policy version changes.

ID	Reader-facing and catalog name	Meaning
CEI-R01	Applicable Evidence (evidence_applicable)	Carries requirement applicability to the exact C4 admission, effect, or Outcome decision.
CEI-R02	Evidence fresh at use (evidence_fresh_at_use)	Carries the Evidence freshness result to the exact C4 consumer decision.
CEI-R03	Observation coverage (observation_coverage)	Carries covered surfaces and detection limits into effect classification.
CEI-R04	Assertion disposition (assertion_disposition)	Carries the evaluator's bounded assertion disposition into effect or Outcome classification.
CEI-R05	Sufficient Evidence (evidence_sufficient)	Carries consumer-specific sufficiency; a failed or unknown result blocks the decision.

ID	Reader-facing and catalog name	Meaning
`CEI-R06`	Bound provenance (`provenance_bound`)	Binds each interface fact to its acquisition and derivation lineage.
`CEI-R07`	Disclosed common causes (`common_causes_disclosed`)	Carries shared dependencies that limit independence and sufficiency claims.

Formal-model actions include package creation, human approval, revocation, human start, reservation, admission or denial at the gateway/backend linearization point, effect observation, response loss, retry request, lease expiry, fence rejection, reconciliation, compensation, Outcome evaluation, checkpoint, restart, and terminal release. Each state-transition model separates safety from bounded resolution-path analysis, represents unavailable or removed assumptions explicitly, uses named events rather than unbounded wall-clock waits, and preserves `unknown` as a first-class state.

Each proposed property requires at least one counterexample produced by removing a named assumption:

Planned property	Named assumption removal	Required counterexample shape
P1 No admission without exact current authority	remove C4-A02 or C4-A03	A bypass writer mutates the target, or a remote backend accepts after a non-atomic pre-call check despite revocation.
P2 Approval binds one canonical package and exact target generation	remove C4-A04 or C4-A18	Target deletion/recreation or canonicalizer disagreement lets a different UID or argument set reuse approval.
P3 Attempt budget and unresolved obligations are monotonic	remove C4-A06 or C4-A17	Checkpoint/restart omits a consumed attempt or admitted unresolved effect and permits another attempt.
P4 Conflicting stale writers cannot both be effective	remove C4-A05 or C4-A07	A lease expires without backend fencing and the old writer acts after a new writer is admitted.
P5 `not_applied` is emitted only with complete applicable observation	remove C4-A11 or C4-A13	An unobserved external controller or side effect exists while the observer reports no change.
P6 Time-invalid authority is not admitted	remove C4-A10	Clock skew exceeds the declared bound and an expired or not-yet-valid decision is accepted.
P7 Outcome success is not inferred from admission or effect	remove C4-A12	The requested mutation occurs but the original Intent criterion fails while the attempt is labeled successful.
L1 Every admitted obligation eventually reaches an authorized terminal disposition	remove C4-A14 or C4-A15	The observer/control plane remains unavailable or an irreversible residual effect has no authorized reconciliation path.

The executable checker uses pinned Go 1.26.10, deterministic exhaustive exploration to depth 8, no random seed, and the finite event sets in `formal/check.go`. On this candidate it visits 1,233 C4 states over 6,014 accepted transitions and 332 Evidence states over 2,325 accepted transitions. `formal/counterexamples.json` retains minimized traces tied to named removed assumptions, including the three required combined traces. A bounded formal state-transition model can falsify a universal claim; successful bounded checking does not prove production safety. Exact acceptance remains an owner decision on the reviewed candidate.

G.8. Public schema and relation-validation slice

This source revision publishes six versioned schemas for `ChangePackage`, `ExecutionRecord`, authority/admission receipt, `Outcome`, immutable C4 lifecycle event, and checkpoint. The schemas reuse the existing `HumanDecision` and `OutcomeCheck` contracts through exact versioned references and canonical digests. `agent-ops.relation-validation@1.0.0` checks five ordered constraint levels: local schema validity; reference and digest integrity; authority and admission; temporal, budget, conflict, and checkpoint invariants; and independent effect, Outcome, and completeness semantics.

Every relation validator has a negative trace whose individual records still pass their JSON Schemas. The corpus rejects package or target substitution, target UID/generation reuse, stale authority epochs, evidence invalid at use, unsafe retry, reservation or fence loss, attempt-budget reset, broken lifecycle linkage, and omitted checkpoint obligations. The relation catalogs pair each service identifier with a reader-facing name, while validator findings expose stable error codes and relation identifiers. Canonicalization is fixed by `agent-ops.c4-json-canonicalization@1.0.0`; `HumanDecision.blocked_action` binds the canonical package approval subject with the two decision-link fields omitted to avoid a circular digest, and a lifecycle link likewise hashes the canonical event with its predecessor-link field omitted.

The validator is an offline checker for this exact record slice. It does not authenticate signatures or provenance, operate the gateway, log, observer, collector, or backend, establish complete lifecycle conformance, evaluate a deployed system, or prove production safety. Runtime implementation, execution evaluation, production deployment, paid evaluation, human-subject research, and release each require the separately applicable decision and evidence.

Appendix H. Evidence Acquisition Profile 1.0.0

H.1. Status, identity, scope, and non-goals

The stable public identity of this profile is `agent-ops.evidence-acquisition@1.0.0`. This appendix is its normative contract for acquisition boundaries, quality axes, failure semantics, trusted computing base, and common-cause treatment and, together with `formal/`, defines the executable relations and transitions. The bounded public record family and validator described in H.8 implement only this named offline slice; they are not a runtime collector or a claim of production conformance.

The profile governs evidence collected for one explicit consumer and use: what must be known, about which exact target and epoch, from which source classes, within which observation window, with what coverage, authority, independence, freshness, negative-observation semantics, and blocking behavior. It neither grants execution authority nor turns successful transport, a digest, a signature, a sealed bundle, or a fluent interpretation into proof of truth or sufficiency.

H.2. Controlled acquisition chain

The controlled chain is:

```
EvidenceRequirement -> CollectionProfile -> CollectionAttempt/Receipt -> EvidenceArtifact
-> EvidenceAssertion -> EvidenceBundle
```

Object	Meaning in this profile
`EvidenceRequirement`	States the consumer, intended claim or decision, exact target and epoch, required sources, time and coverage bounds, authority, independence, freshness, negative-observation semantics, and the result of missing or conflicting evidence.
`CollectionProfile`	Versioned and digest-bound collector contract: tool and configuration, permissions and read-only boundary, source and target binding, population or sampling and seed, limits, retry and error rules, parser, normalization, redaction, raw references, coverage, common-cause dependencies, and reproducibility inputs.

Object	Meaning in this profile
`CollectionAttempt/Receipt`	Immutable record of every attempt, including failures: request and query digest, transport and parser states, retries and interruption, actual coverage, included and excluded items, counts, truncation, dropped data, and transformations. A retry creates a new receipt.
`EvidenceArtifact`	Exact retained bytes or protected reference bound to one receipt, with content identity and transformation lineage.
`EvidenceAssertion`	A scoped statement that an artifact `supports`, `refutes`, `does_not_test`, `conflicts`, or leaves `unknown` a named claim. It distinguishes an observed negative from missing data.
`EvidenceBundle`	A content-addressed package of references and lineage. Packaging or sealing creates neither authority nor evidence quality. Existing Bundle v1 semantics remain unchanged; the incompatible acquisition contract uses the separate Bundle v2 schema identity.

Each link MUST preserve the identities and limitations of the preceding link. A derived representation cannot silently widen scope, improve trust class, erase exclusions, replace an exact target, or convert `unknown` into success.

H.3. Independent quality axes and claim rule

Quality axis	Required question
Authority	Is the source recognized for this exact fact, scope, and time?
Identity and binding	Are consumer, target, source, epoch, and artifact unambiguously bound?
Integrity and provenance	Can the examined bytes and every transformation be traced without an unexplained gap?
Acquisition validity	Did the declared collector, query, permissions, bounds, parser, and error rules actually govern the attempt?
Coverage and completeness	What population, interval, surfaces, inclusions, exclusions, truncation, and dropped data were actually covered?
Freshness	Is the evidence fit at use time for the target and every decision-critical version?
Transformation fidelity	Did parsing, normalization, aggregation, redaction, or projection preserve the claim-relevant meaning and limitations?
Independence	Which sources, credentials, collectors, parsers, stores, clocks, paths, language models, or prompts are shared?
Claim support	Does the assertion support or refute the exact claim, merely fail to test it, conflict, or remain unknown?
Reproducibility	Can a reviewer recover the inputs, profile, receipts, artifacts, transformations, and bounded result?

The consumer evaluates sufficiency against its own `EvidenceRequirement`; evidence is not globally sufficient. Every required axis MUST be explicit. Missing, conflicted, failed, or unverified blocking axes yield `unknown` or incomplete. An aggregate score MUST NOT compensate for failed authority, identity binding, integrity, or blocking coverage. Evidence may justify a factual claim only within the evaluated boundary; it MUST NOT grant approval, execution authority, or source authority.

H.4. Trusted computing base

Component	Required responsibility and claim limit
Source authority	Establishes which source may speak for which fact; does not prove the source correct.
Collector or tool	Executes the bound read contract and exposes failures and limits; successful transport does not prove completeness or truth.

Component	Required responsibility and claim limit
Target/source identity service	Resolves exact identities and epochs; aliases alone do not prove binding.
Time/version source	Supplies bounded time and decision-critical versions; shared or uncertain sources limit freshness claims.
Raw evidence store	Retains exact artifacts or protected references; storage does not create authority.
Transformation pipeline	Parses, normalizes, aggregates, and redacts with recorded lineage; it cannot improve source quality.
Canonicalizer and digest function	Bind canonical bytes under one accepted algorithm and version; they do not prove semantics.
Bundle writer	Packages references and limitations without rewriting them; sealing does not establish sufficiency.
Relation validator	The published C4 validator checks only its accepted cross-record slice; it cannot infer missing facts or authority, and no general Evidence Acquisition validator is claimed.
Evaluator or oracle	Applies the consumer requirement to assertions; its correctness and coverage remain named assumptions.

H.5. Failure and negative-evidence semantics

Transport failure, permission denial, timeout, parser failure, truncation, sampling shortfall, redaction loss, source conflict, and target or version mismatch are distinct outcomes and MUST remain recorded. A successful request or non-empty artifact does not erase them. A retry is another attempt with another receipt; it does not rewrite the prior attempt.

`not_observed` is evidence of absence only when the requirement defines the population and window and the receipt demonstrates complete applicable coverage and detection capability. Otherwise the assertion is `does_not_test` or `unknown`, not a negative fact. Conflicting authoritative sources remain conflicted until a declared resolution rule is applied; selection by convenience is not resolution.

H.6. Interface with the limited C4 profile

C4 MUST use this profile when it claims knowledge of an actual effect, including `not_applied`; classifies Outcome against the original Intent; or claims evidence sufficiency at admission or use time. The C4 effect observer and Outcome oracle act as evidence producers and consumers under explicit requirements. Their receipts, coverage, detection capability, freshness, assertion disposition, and common-cause references constrain C4-G05 and C4-G06.

The profiles remain independent. Evidence never grants approval or execution authority, and C4 admission authority never proves evidence true, complete, current, or sufficient. A validly authorized operation may have `unknown` effect or Outcome; a well-supported observation may describe an unauthorized operation without authorizing it. `agent-ops.c4-evidence-interface@1.0.0` carries requirement applicability and consumer, exact target and epoch, use-time freshness with decision-critical versions, observation coverage and detection capability, assertion disposition, consumer-specific sufficiency, provenance binding, and common-cause disclosure. These interface relations (`CEI-R01-CEI-R07`) are mapped individually in the G.7 relation index. The bounded C4 `ExecutionRecord` serializes the facts consumed by this interface; it is not a standalone Evidence Acquisition record family or validator.

H.7. Common-cause boundaries and counterexamples

An implementation record MUST enumerate shared sources, credentials, collectors, parsers, storage, clocks, network paths, language models, prompts, canonicalizers, identity services, and target APIs. Two assertions derived through one defective dependency are not independent evidence of that dependency. Different process, language-model, or prompt names do not establish independence when the authority, data, credentials, or observation path remains shared.

The executable specification retains minimized counterexamples showing at least: a TTL-valid artifact after a decision-critical version change; `not_applied` inferred from incomplete coverage; a successful transport with truncated or unparsable content; a sealed bundle with an unauthorized source; two apparently independent assertions sharing one false source; a correct effect observation incorrectly treated as Outcome success; and valid C4 authority incorrectly treated as evidence truth. They are executable entries in `formal/counterexamples.json`.

The Evidence checker names the following properties:

Property	Required invariant
E1	Freshness requires exact target, epoch, and decision-critical version binding at use.
E2	A supported negative assertion requires complete applicable coverage and detection capability.
E3	Transport, parse, truncation, sampling, or transformation loss remains visible and blocks sufficiency.
E4	An unauthorized source cannot produce a sufficient or sealed result.
E5	Shared or undisclosed common causes cannot be counted as independent evidence.
E6	Attempts, receipts, failures, conflicts, and pending obligations are append-only across checkpoint and restart.
E7	C4 approval or admission authority cannot establish evidence truth or sufficiency.

H.8. Candidate boundary

This candidate defines the executable formal Evidence Acquisition state-transition model, composes it through `agent-ops.c4-evidence-interface@1.0.0` with the separate formal C4 state-transition model, and publishes `agent-ops.evidence-relations@1.0.0`. The deterministic checker record and retained counterexamples are specified in G.7 and `formal/README.md`.

The bounded public implementation adds `EvidenceRequirement`, `CollectionProfile`, `CollectionReceipt` with its artifact record, and `EvidenceAssertion` schemas at `1.0.0`, the separate `evidence_bundle_v2.schema.json` identity at `2.0.0`, and `agent-ops.evidence-relation-validation@1.0.0`. Its result reports local schema validity, cross-record relation validity, and consumer-specific sufficiency separately. The fixture corpus pairs every negative case with a valid baseline and covers empty or misbound completion, hidden acquisition loss, wrong targets or epochs, stale critical versions, broken lineage, invalid negative observations, false independence, hidden conflicts, digest mismatch, and lost history.

Evidence Bundle v1 remains valid only under its original local contract. Migration creates new requirement, profile, receipt/artifact, assertion, and v2 bundle records; it never rewrites v1 bytes or promotes v1 `present`, digest, or `sealed` fields to v2 completeness or sufficiency. Missing attempt, exclusion, partial, unknown, or conflict history makes migration insufficient. The validator does not authenticate sources, collect evidence, grant execution authority, establish general truth, or prove production safety.

H.9. Integrated conformance and pre-runtime evaluation protocol

The composed offline profile `agent-ops.c4-evidence-integrated@1.0.0` pins the exact C4 and Evidence validators and the C4-Evidence interface. Its result reports local syntax, C4 relation validity, Evidence relation validity, consumer-specific Evidence sufficiency, cross-boundary validity, and the consuming decision separately. `allowed` requires every component result and the interface projection to pass; malformed input remains `unknown`, while a complete but invalid input is `blocked`. The composed checker cannot change a component relation or reason code, and an aggregate score cannot override a blocking finding.

The cross-boundary corpus pairs each negative trace with a valid baseline. It covers an Executor response used as effect proof, `not_applied` under partial coverage, Outcome success from stale or insufficient Evidence, retry from an incomplete negative observation, and checkpoint plus supersession hiding acquisition failure and an unresolved effect. Property tests verify order invariance, monotonic propagation of `unknown`, malformed-input

safety, and exact catalog pins. The C4 `consumer_rule` fields use the same explicit versioned identifier syntax as Evidence Acquisition; this corrects the earlier non-composable candidate constraint and changes the pinned bytes of the two affected C4 schemas without changing their resource identities.

The pre-runtime evaluation and reproducibility protocol `agent-ops.c4-evidence-evaluation@1.0.0` freezes all assigned cases, deterministic seed `30404`, named barriers, exact tool/model/prompt/profile/schema/catalog versions, target and source snapshots, authority and expected-coverage maps, failure injection, reset and adjudication records, and the required raw and derived artifacts. `succeeded`, `failed`, `timeout`, `partial`, and `unknown` all remain in the denominator. Primary metrics separately cover unauthorized or duplicate effects, clean and adversarial task success, checkpoint violations, false completeness, unsupported assertions, wrong target or source, stale or hidden-loss Evidence, broken lineage, false independence, invalid negative evidence, and snapshot/method reproducibility.

The protocol records how a future separately authorized evaluation MUST be measured before results are inspected. It reports no collector or Executor run, pilot, deployment, runtime frequency, production-safety result, universal Evidence truth, or release readiness. A finite synthetic corpus may expose a defect or falsify a bounded claim; it cannot establish those broader claims.

> *Note: this white paper describes the methodology and does not replace normative machine-readable schemas, specifications, policies, or implementation documentation.*